

大模型技术及开发

大模型的基石：Transformer

主讲人：陈小军

时间：2025.3.21

目录

CONTENTS

01 | RNN

02 | Transformer

03 | Transformer改进

04 | 总结与讨论

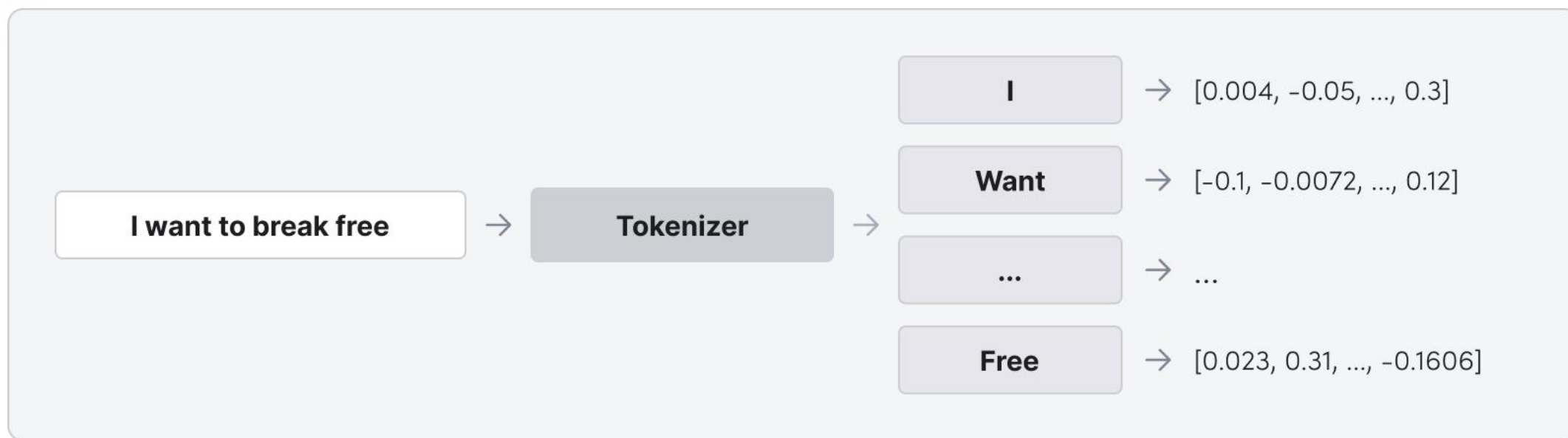


01

RNN

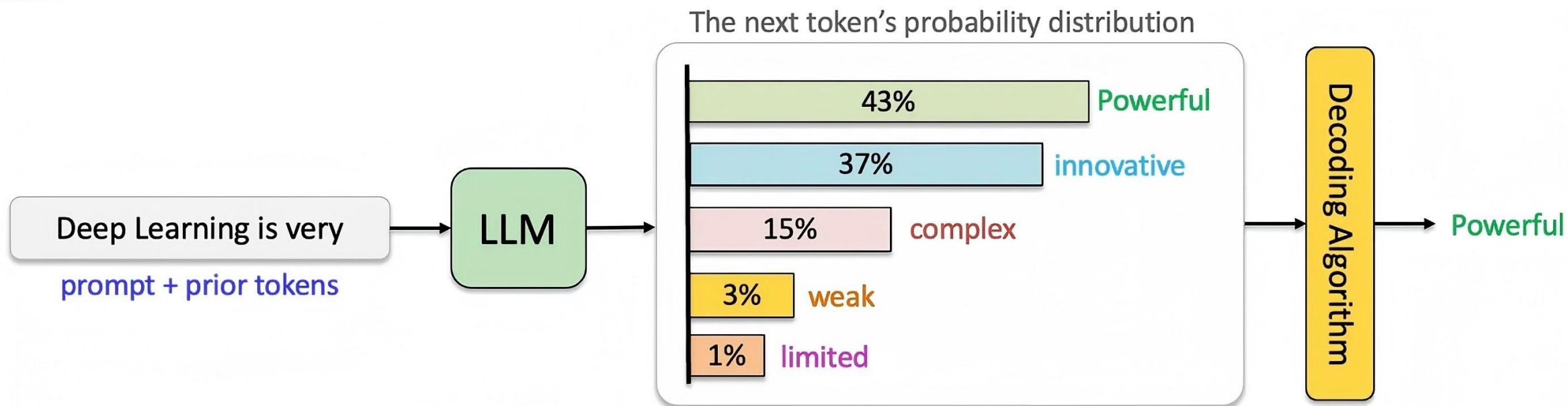
2025

Token



输入和输出文本都会被分解成称为tokens（标记）的片段。
每个token都有一份关联的权重列表，称为vector（向量）。例如：向量中的每个值（权重）代表标记特定方面的程度——例如，“肤色”、“年龄”或“家庭关系”，或者其他没有简单术语，

自回归模型 (Autoregressive Models)



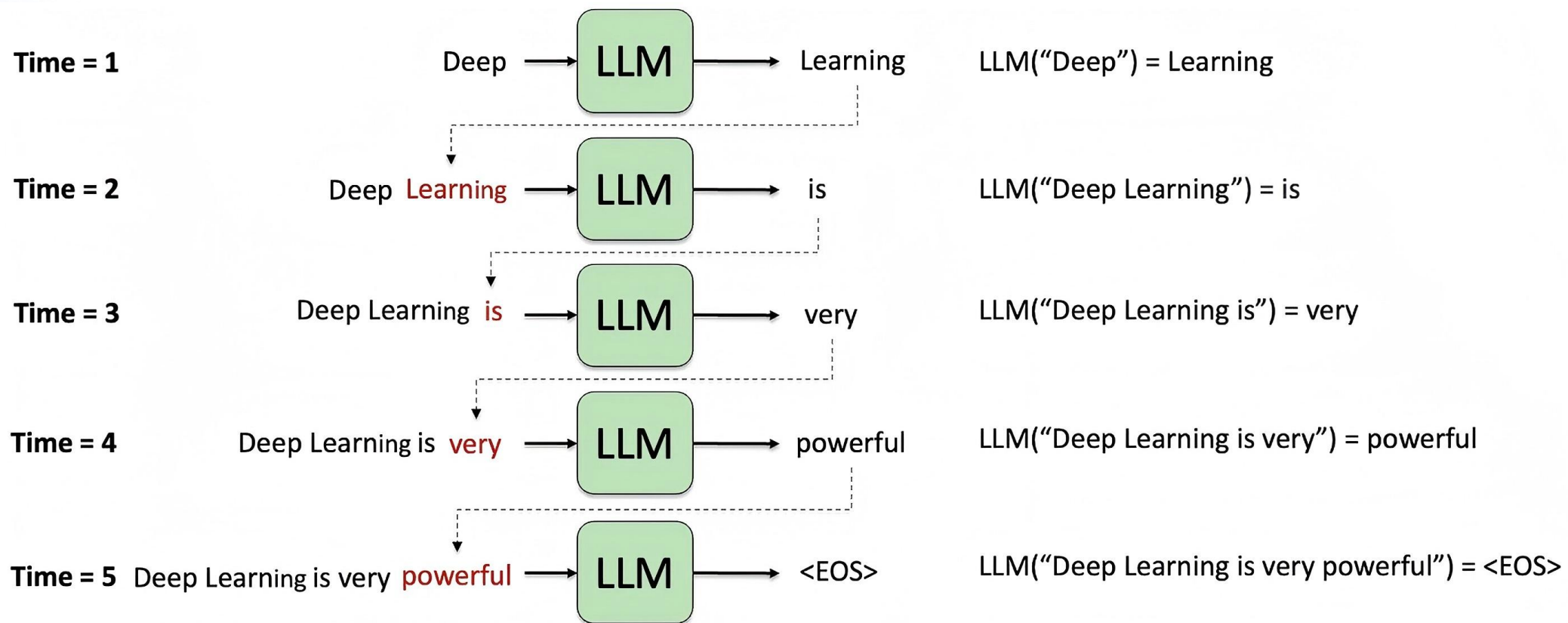
自回归模型：根据前面的**token**预测下一个**token**（或token / sub-word）的**概率分布**。这种自回归特性使模型能够学习复杂的语言模式和依赖关系，从而善于生成。

在数学上，LLM 是一个概率模型(Probabilistic Model)，学习如下概率：

$$p(w_t | w_1, \dots, w_{t-1})$$



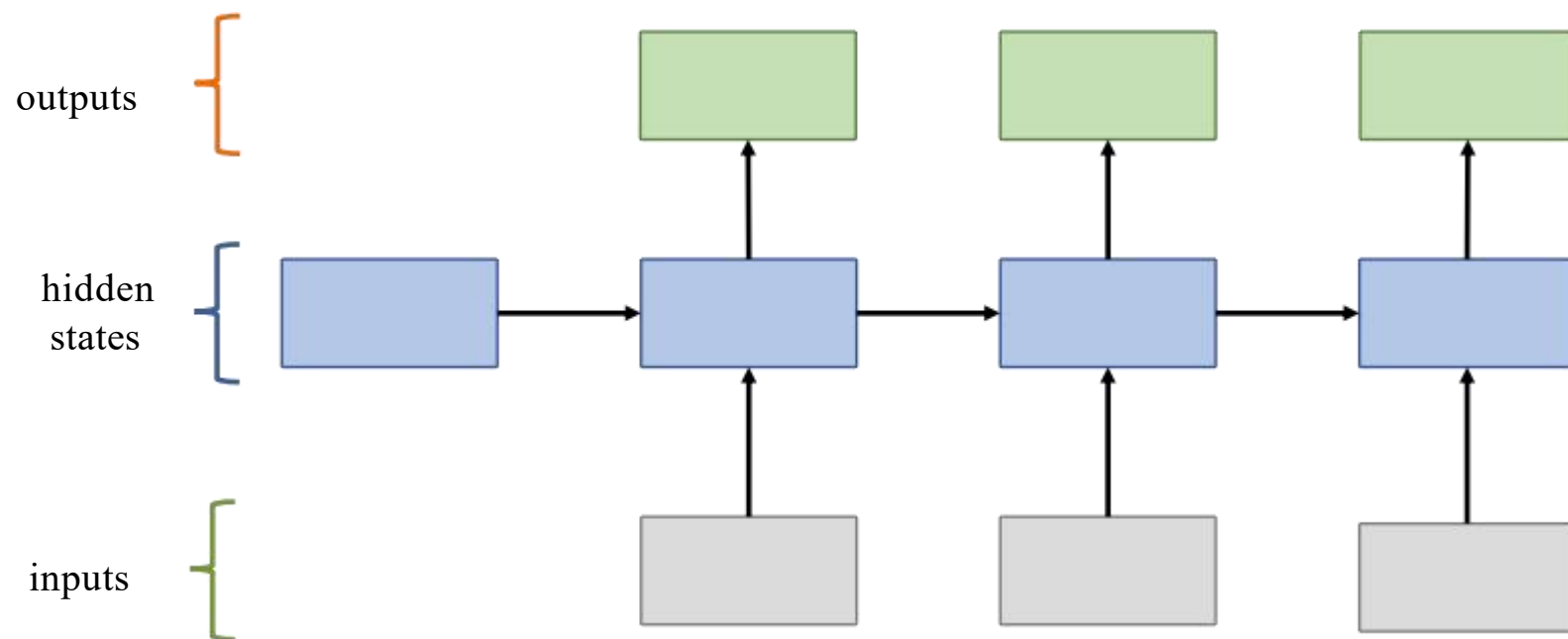
自回归模型生成-接龙游戏



在生成时，LLM通过「**解码算法**」([Decoding Algorithm](#))，从一个提示开始，逐步确定下一个输出的token。这一过程可以采用不同的策略：既可以选择概率最高的下个字（即贪婪搜索），也可以从预测的概率分布中**随机采样一个token**，这个随机方法使得每次生成的文本都可能有所不同，**这种特性与人类语言的多样性和随机性颇为相似。**

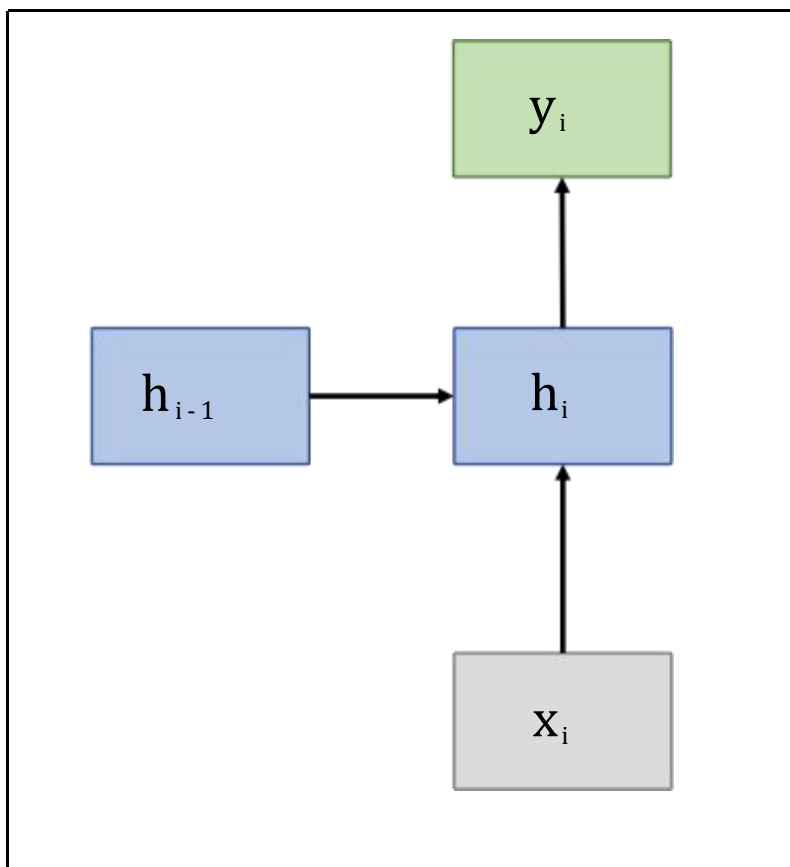


RNN (循环神经网络)





RNN (循环神经网络)



$$h_i = \tanh(w_x x_i + w_h h_{i-1} + b)$$

$$y_i = g(h_i)$$

g 是分类函数，将隐变量与字符库关联起来

基于RNN的语言模型

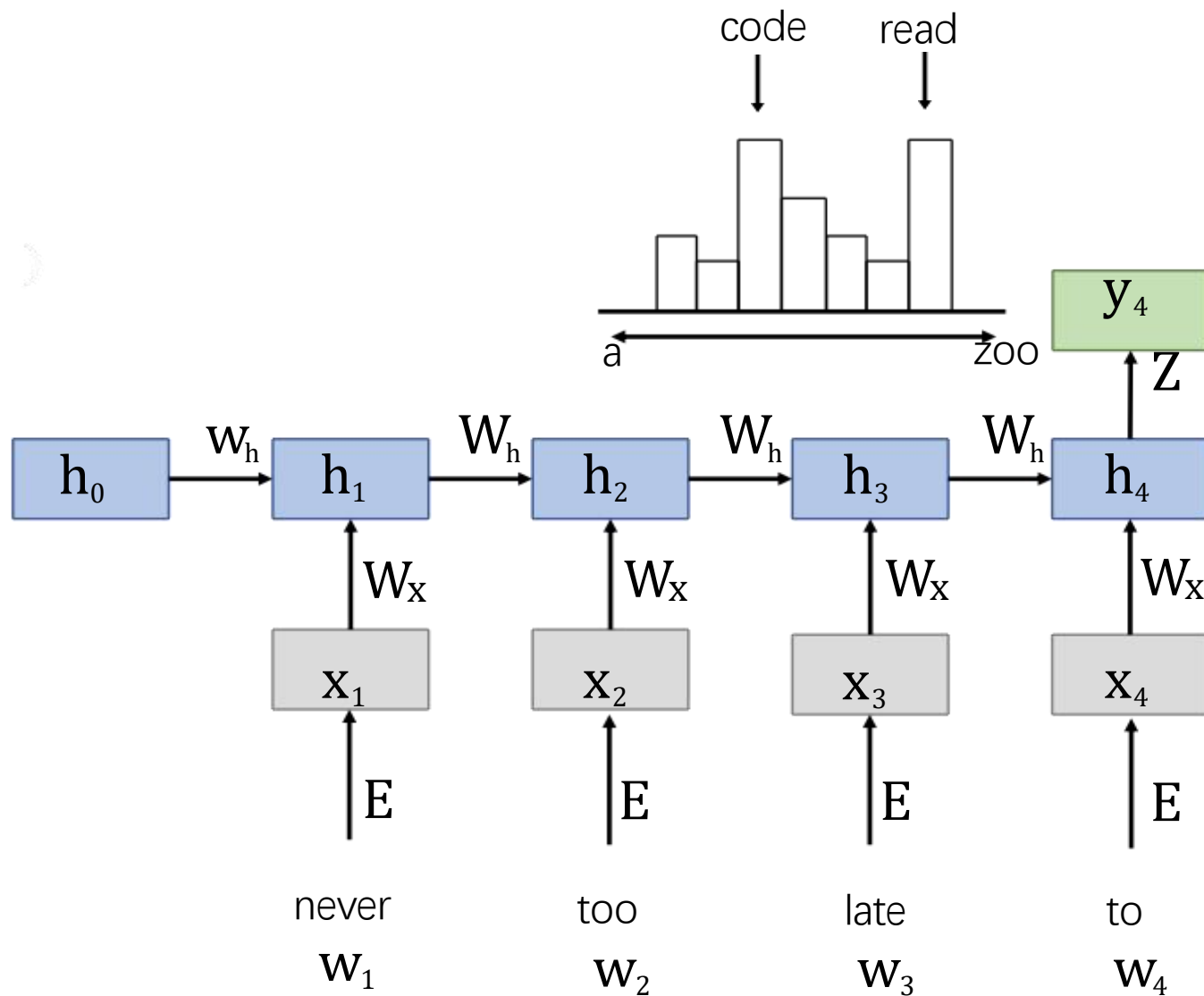
对词序列 $\{w_1, w_2, w_3, \dots, w_N\}$, 基于 RNN 的语言模型每次根据当前词 w_i 和循环输入的隐藏状态 h_{i-1} , 来预测下一个词 w_{i+1} 出现的概率, 即

$$P(w_{i+1}|w_{1:i}) = P(w_{i+1}|w_i, h_{i-1}) = g(h_{i-1})$$

$$h_i = f(x_i, h_{i-1}) \quad h_0 = \{0, \dots, 0\}$$

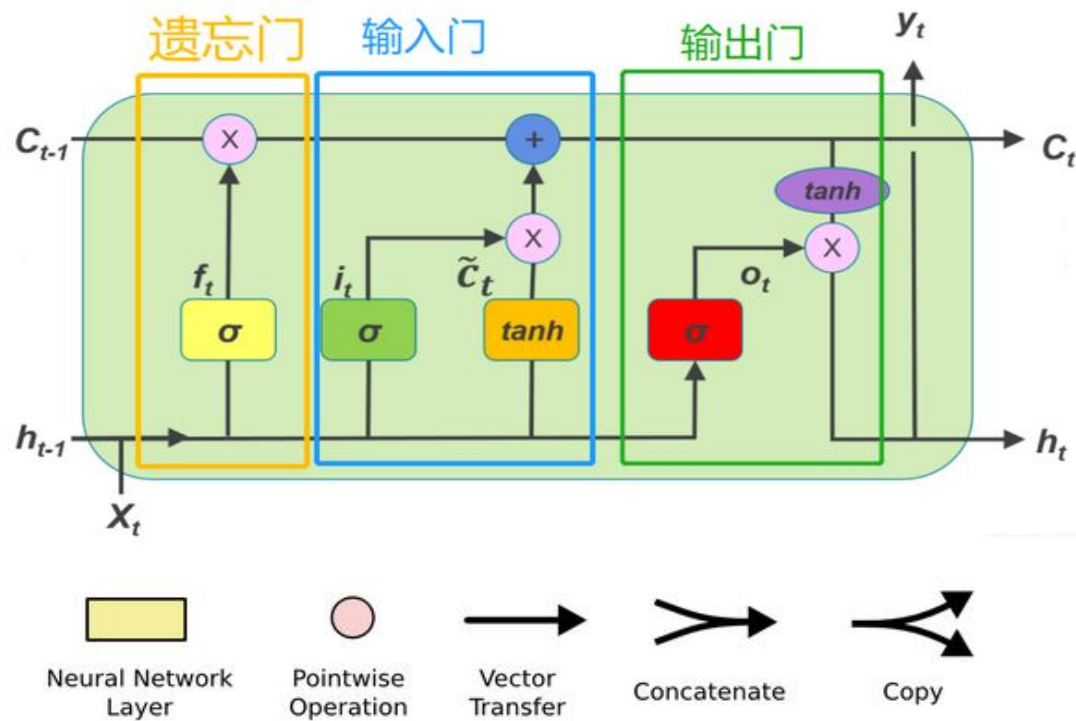
f 是原始的RNN或其扩展模型

$$y_i = g(h_i) \quad g \text{ 是分类函数, 如softmax}$$



缺点: 不能够并行计算、梯度消失或爆炸 (梯度随着层数可能快速缩小或放大)、难以捕捉长距离关系 (信息随着距离的增加影响力急剧下降)。

RNN扩展: LSTM



遗忘门 (Forget Gate)

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

输入门 (Input Gate)

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

细胞状态更新 (Cell State Update)

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

输出门 (Output Gate)

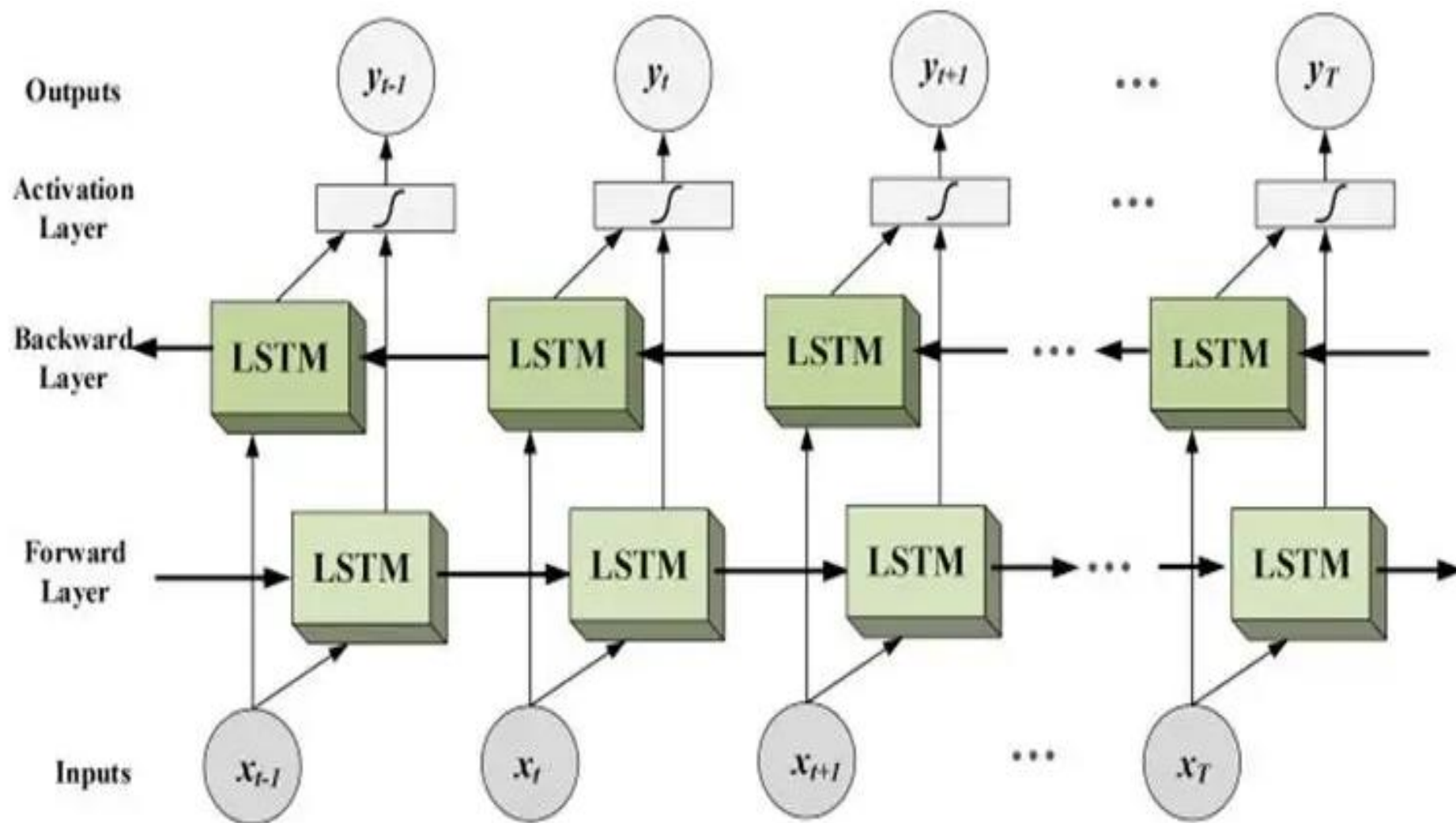
$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

隐藏状态更新 (Hidden State Update)

$$h_t = o_t \cdot \tanh(C_t)$$

缺点：训练效率较低（无法并行化处理）、长期依赖性处理能力较弱（虽然较RNN有优势，但问题仍然存在）、模型深度受限（于循环结构限制堆叠过多的 LSTM 层会导致梯度消失或梯度爆炸问题）、难以捕捉全局信息（信息距离过远会衰减很快）。

▶ 基于双向LSTM的语言模型

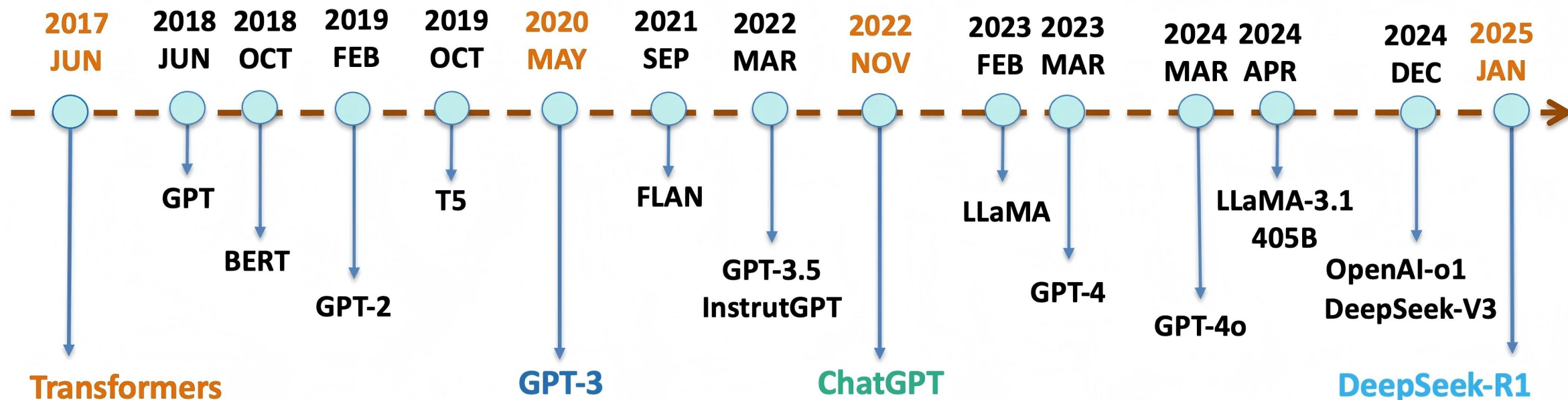


02

Transformer

2025

基于Transformer的大语言模简史



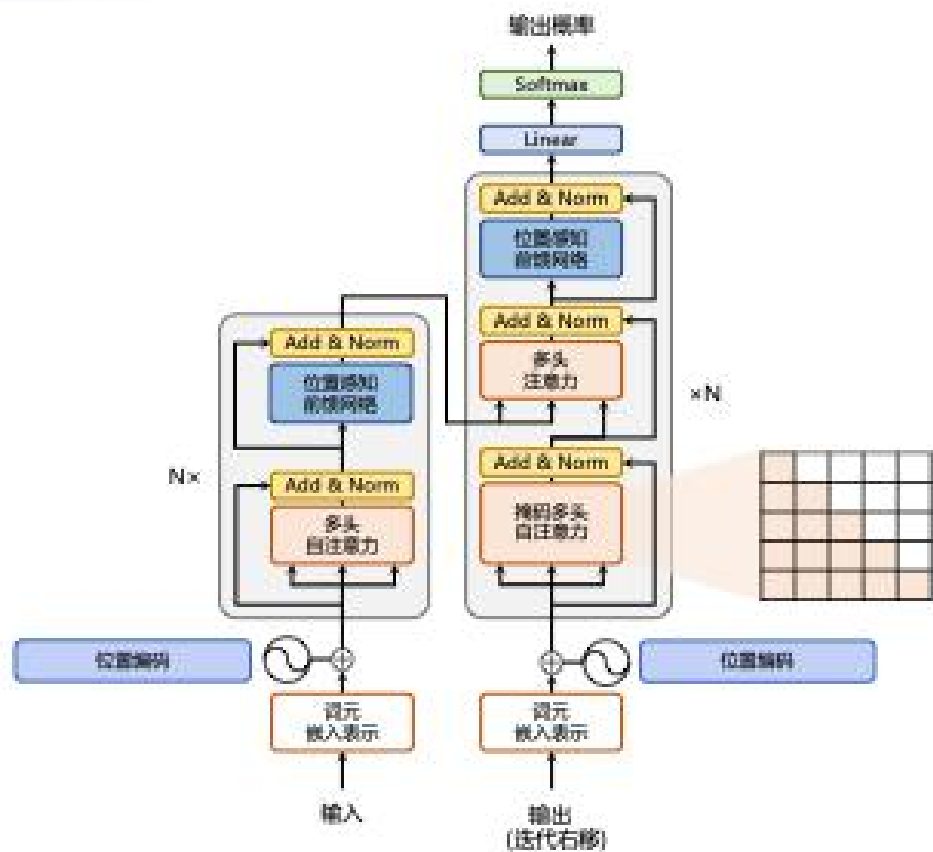
Transformer架构诞生，奠定了现代大语言模型发展的基石。

GPT-3的问世，引领大语言模型迈入规模化与通用化的新纪元。

ChatGPT的问世，开创了大语言模型走向大众、融入日常生活的全新时代。

DeepSeek的入场，为大语言模型带来低成本与开源的甘霖，加速技术平民化进程。

Transformer



- 左侧和右侧分别对应着编码器（Encoder）和解码器（Decoder）结构，它们均由若干个基本的Transformer 块（Block）组成
- 每个Transformer 块都接收一个向量序列 $\{x_i\}$ 作为输入，并输出一个等长的向量序列 $\{y_i\}$ 作为输出
- y_i 是当前Transformer 块对输入 x_i 进一步整合其上下文语义后对应的输出
- 多头注意力（Multi-Head Attention）：多个注意力网络整合上下文语义，使得序列中任意两个单词之间的依赖关系可以直接被建模，从而更好地解决文本的长程依赖问题。
- 位置感知前馈层（Position-wise FFN）：通过全连接层对输入文本序列中的每个单词表示进行更复杂的变换。
- 残差连接：图中的Add 部分，是一条分别作用在上述两个子层中的直连通路，被用于连接两个子层的输入与输出，使信息流动更高效，有利于模型的优化。
- 层归一化：图中的Norm 部分，对表示序列进行层归一化操作，同样起到稳定优化的作用。

- 2017年，Vaswani等人通过其开创性论文《Attention is All You Need》引入了Transformer架构
- Transformer已经成为现代大型语言模型的核心架构，同时成功应用到其他类型数据上。



创新技术

问题

长距离依赖问题

序列建模问题

非线性关系建模问题

上下文泄露
(Context Leakage)

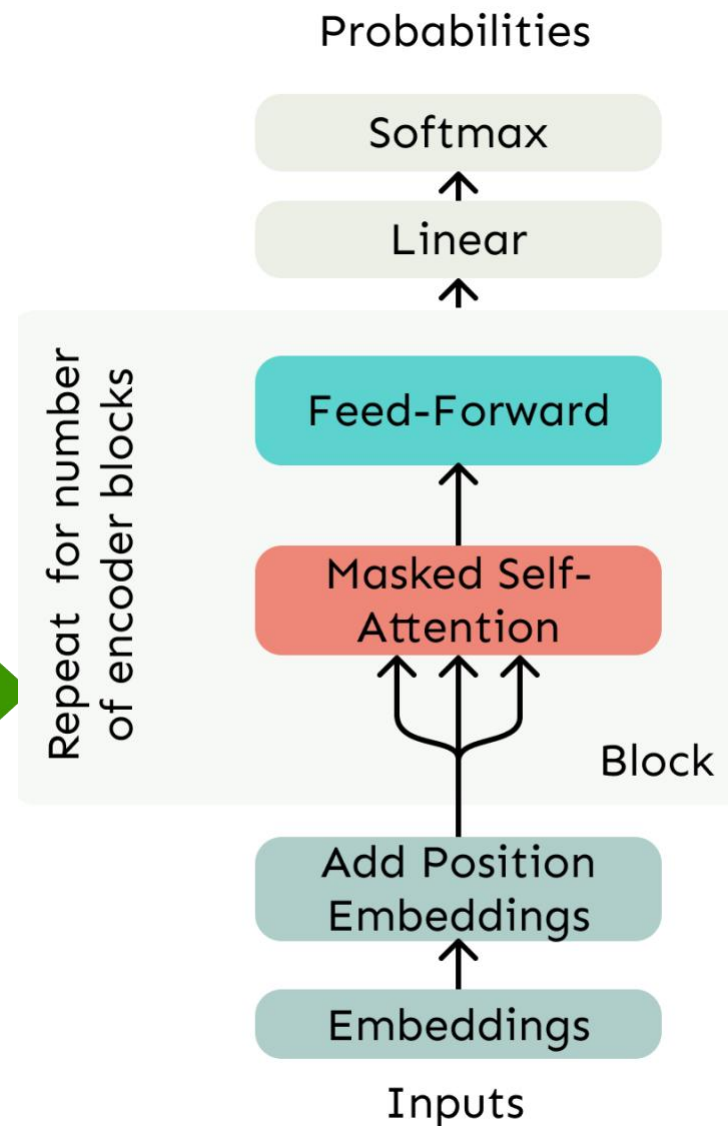
方案

注意力机制

位置编码

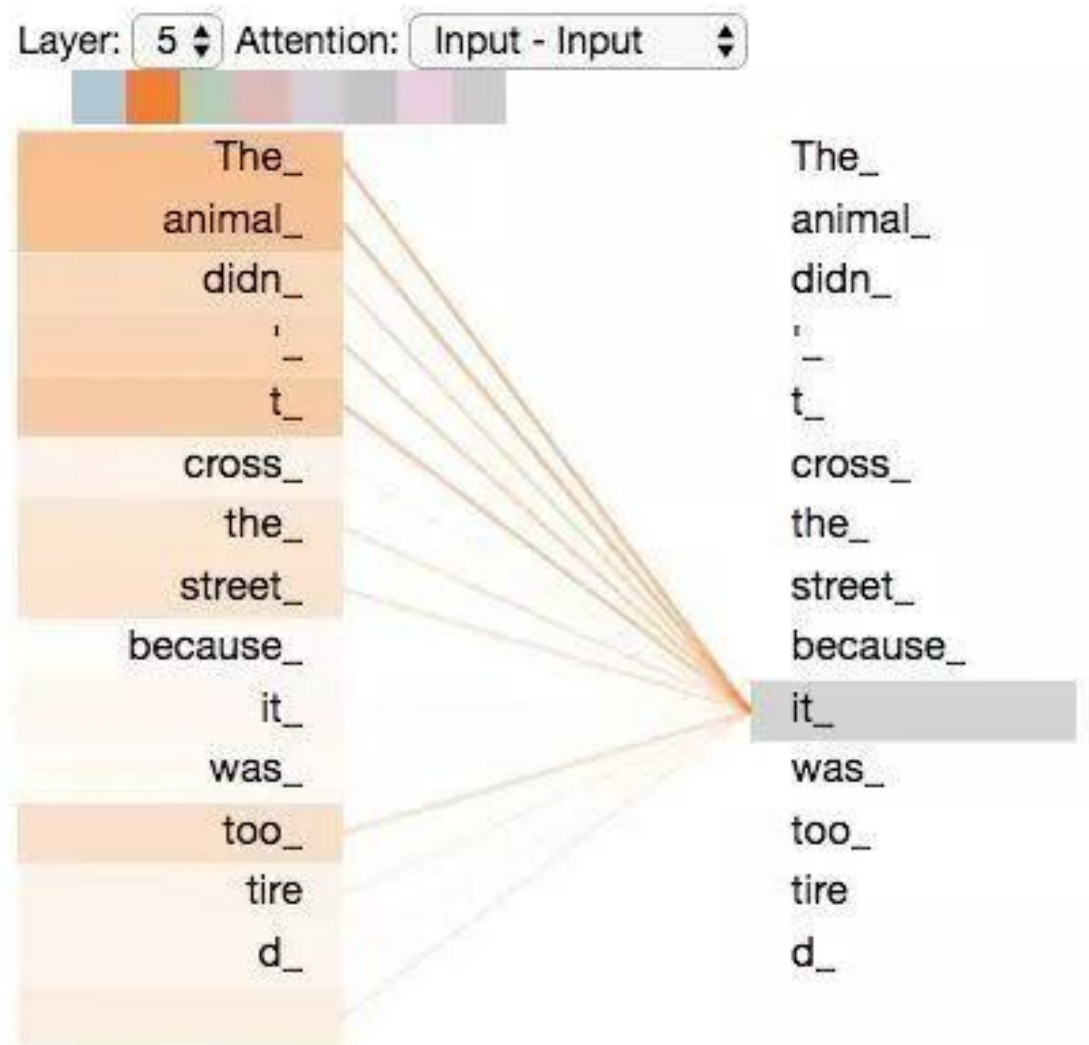
前馈网络

掩码自注意力机制



自注意力机制意义

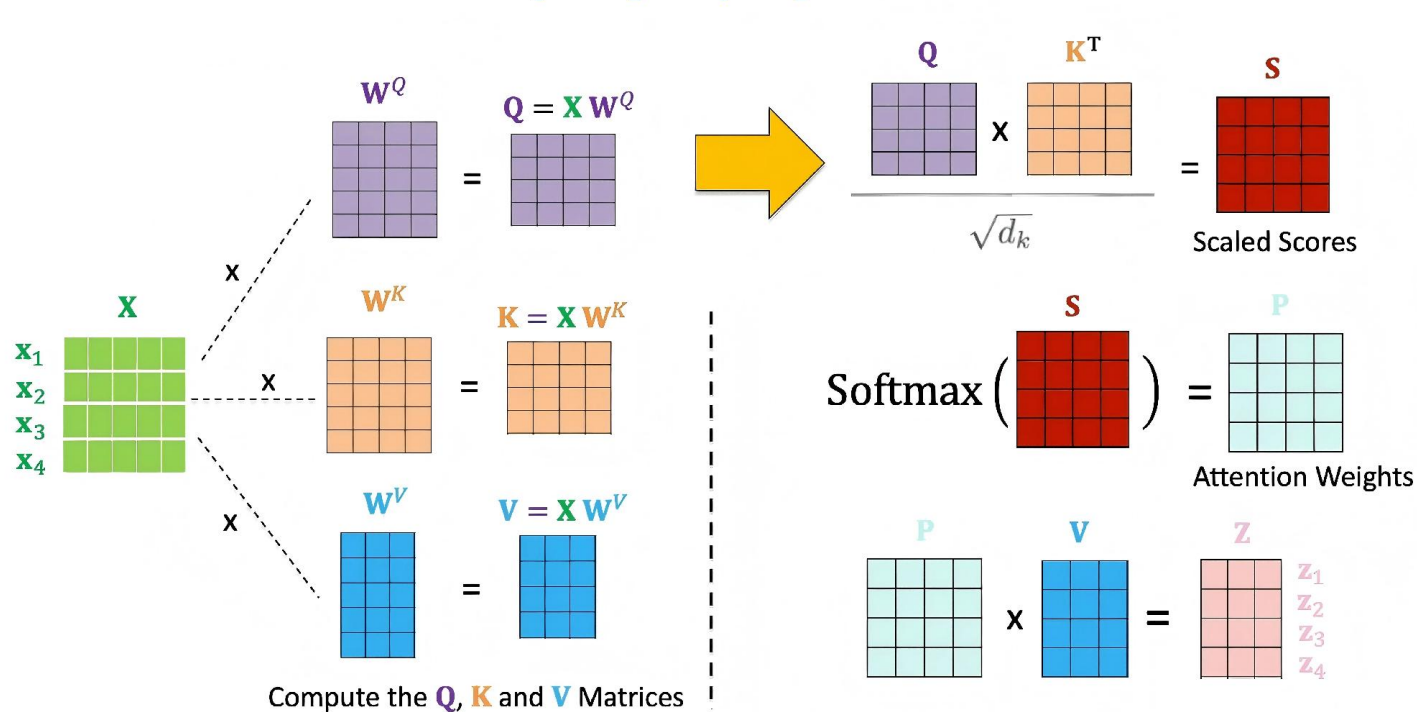
随着模型处理输入序列的每个单词，自注意力会计算整个输入序列的所有单词与某个单词的相关性，帮助模型理解单词之间的关系。



当编码器#5（栈中最上层编码器）中编码“it”这个单词的时，注意力机制的部分会去关注“The Animal”，将它的表示的一部分编入“it”的编码中。

自注意力机制 (Self-Attention) 计算方法

Self-Attention: **X**, **Q**, **K**, **V**, **Z** Matrices



$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$

Q: query, 要查询的信息

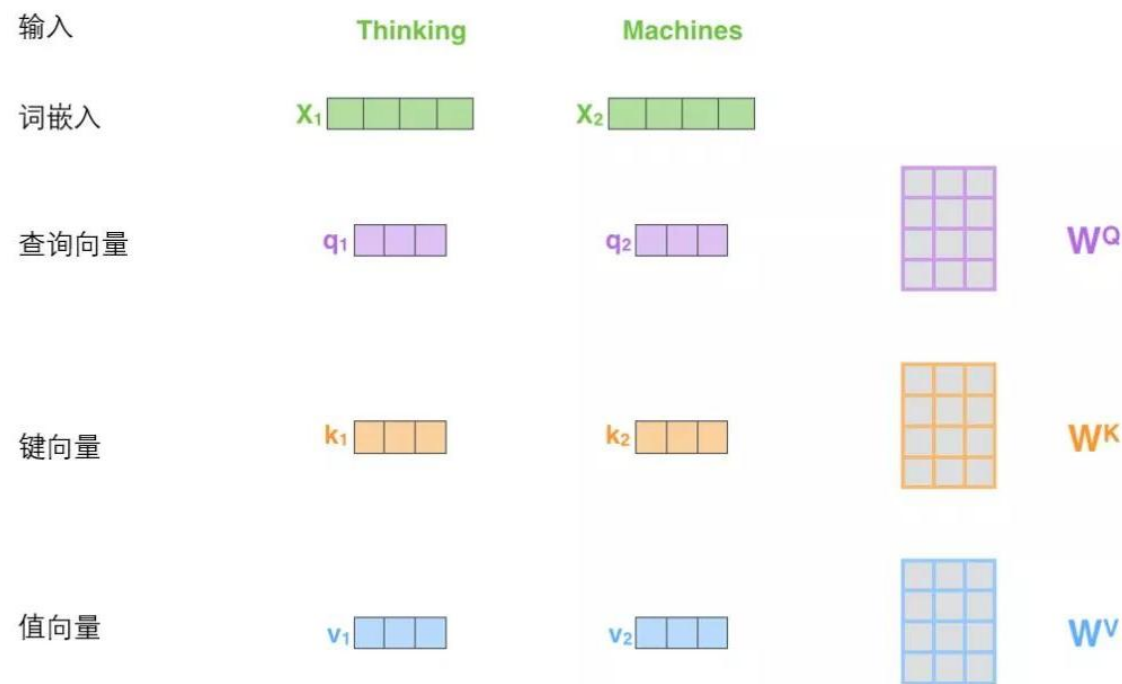
K: key, 等着被查的

V: value, 实际的特征信息

其中, **Q**、**K**、**V**是查询(query)、键(key)和值(value)矩阵, d_k 是键向量的维度。自注意力机制不仅支持并行计算, 显著加快了训练速度, 还增强了模型对全局上下文的理解能力。

自注意力机制计算原理

计算自注意力的第一步就是从每个编码器的输入向量（每个单词的词向量）中生成三个向量。也就是说对于每个单词，创建一个查询向量(Q)、一个键向量(K)和一个值向量(V)。这三个向量是通过词嵌入与三个权重矩阵后相乘创建的，它们的维度是64，而词嵌入和编码器的输入/输出向量的维度是512。但实际上不强求维度更小，这只是一直基于架构上的选择，它可以使多头注意力（multiheaded attention）的大部分计算保持不变。



x_1 与 W_Q 权重矩阵相乘得到 q_1 , 就是与这个单词相关的查询向量。最终使得输入序列的每个单词的创建一个查询向量Q、一个键向量K和一个值向量V。

▶ 自注意力机制计算原理

什么是查询向量Q、键向量K和值向量V?

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

计算得分

分数除以8，然后通过softmax传递结果。

将每个值向量乘以softmax分数(这是为了准备之后将它们求和)。

对加权值向量求和，然后即得到自注意力层在该位置的输出。

输入

词嵌入

查询向量

键向量

值向量

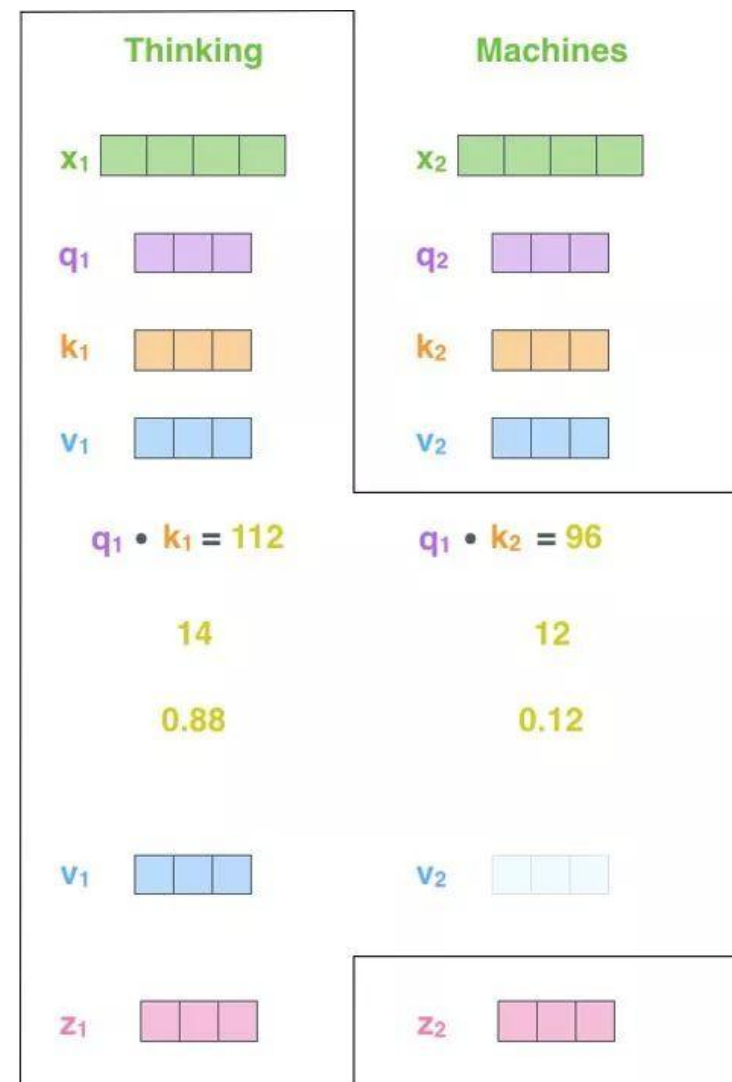
打分

除以8 ($\sqrt{d_k}$)

Softmax

softmax
乘以
值向量

求和



自注意力机制计算原理

通过矩阵运算实现自注意力机制

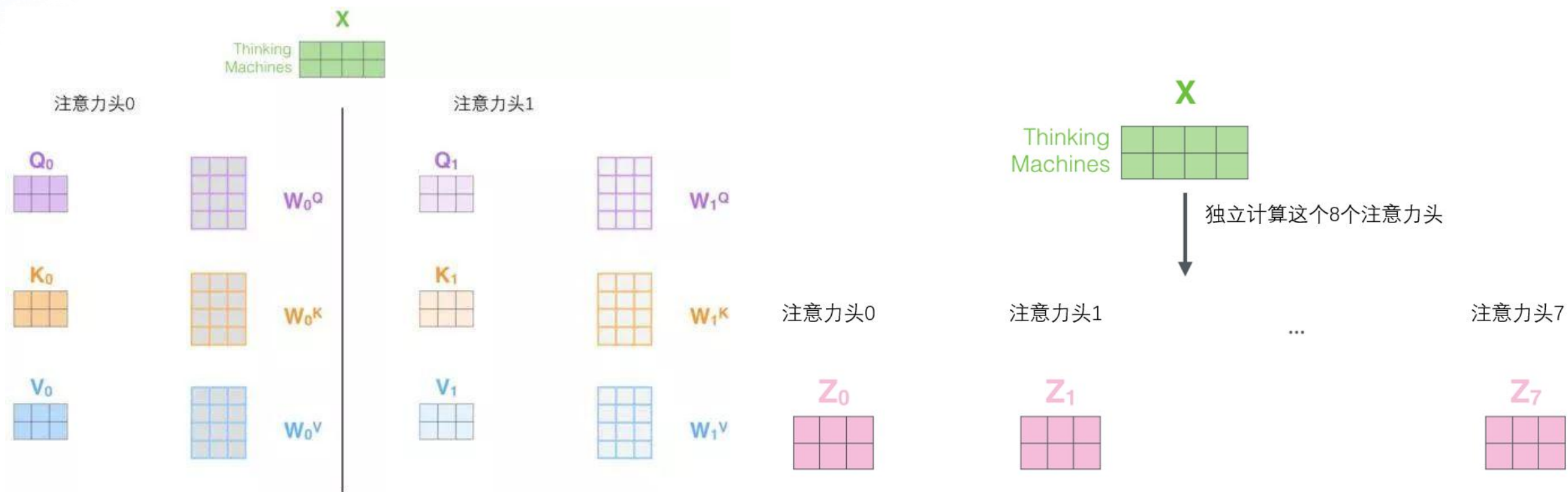
第一步是计算查询矩阵、键矩阵和值矩阵。为此，将输入句子的词嵌入装进矩阵X中，将其乘以我们训练的权重矩阵(W^Q , W^K , W^V)。

x矩阵中的每一行对应于输入句子中的一个单词。我们再次看到词嵌入向量 (512, 或图中的4个格子)和 q/k/v向量(64, 或图中的3个格子)的大小差异。

最后，由于我们处理的是矩阵，我们可以用一个公式来计算自注意力层的输出。

$$\begin{aligned} X &\times W^Q = Q \\ X &\times W^K = K \\ X &\times W^V = V \\ \text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) &\times V = Z \end{aligned}$$

▶ 多头注意力 (Multi-Head Attention)



一组Q,K,V得到了一组当前词的特征表达

▶ 多头注意力 (Multi-Head Attention)

1) 将所有注意力头拼接起来



2) 乘以矩阵 W^O , 它在模型中是联合训练的

$\times$



3) 结果是一个融合所有注意力头信息的矩阵 Z , 我们可以将其送到前馈神经网络



可以看到 Multi-Head Attention 输出的矩阵 Z 与其输入的矩阵 X 的维度是一样的。



多头注意力 (Multi-Head Attention)

1) 这是我们的输入句子*

2) 编码每一个单词

3) 将其分为8个头，将矩阵X或R乘以各个权重矩阵

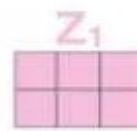
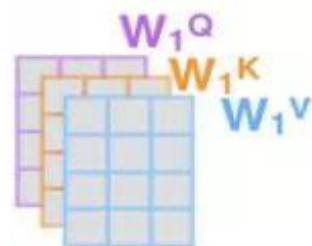
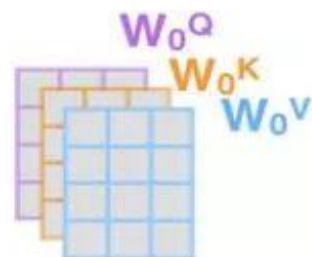
4) 通过输出的查询/键/值 (Q/K/V) 矩阵计算注意力

5) 将所有注意力头拼接起来，乘以权重矩阵 W^O

Thinking
Machines



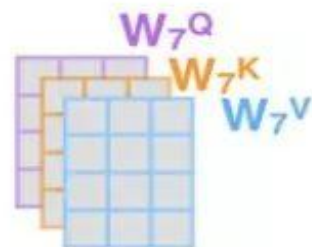
*注：除了第0个编码器，其他编码器都不需要进行词嵌入。它可以直接讲前面一层编码器的输出作为输入（矩阵R）。



...

...

...

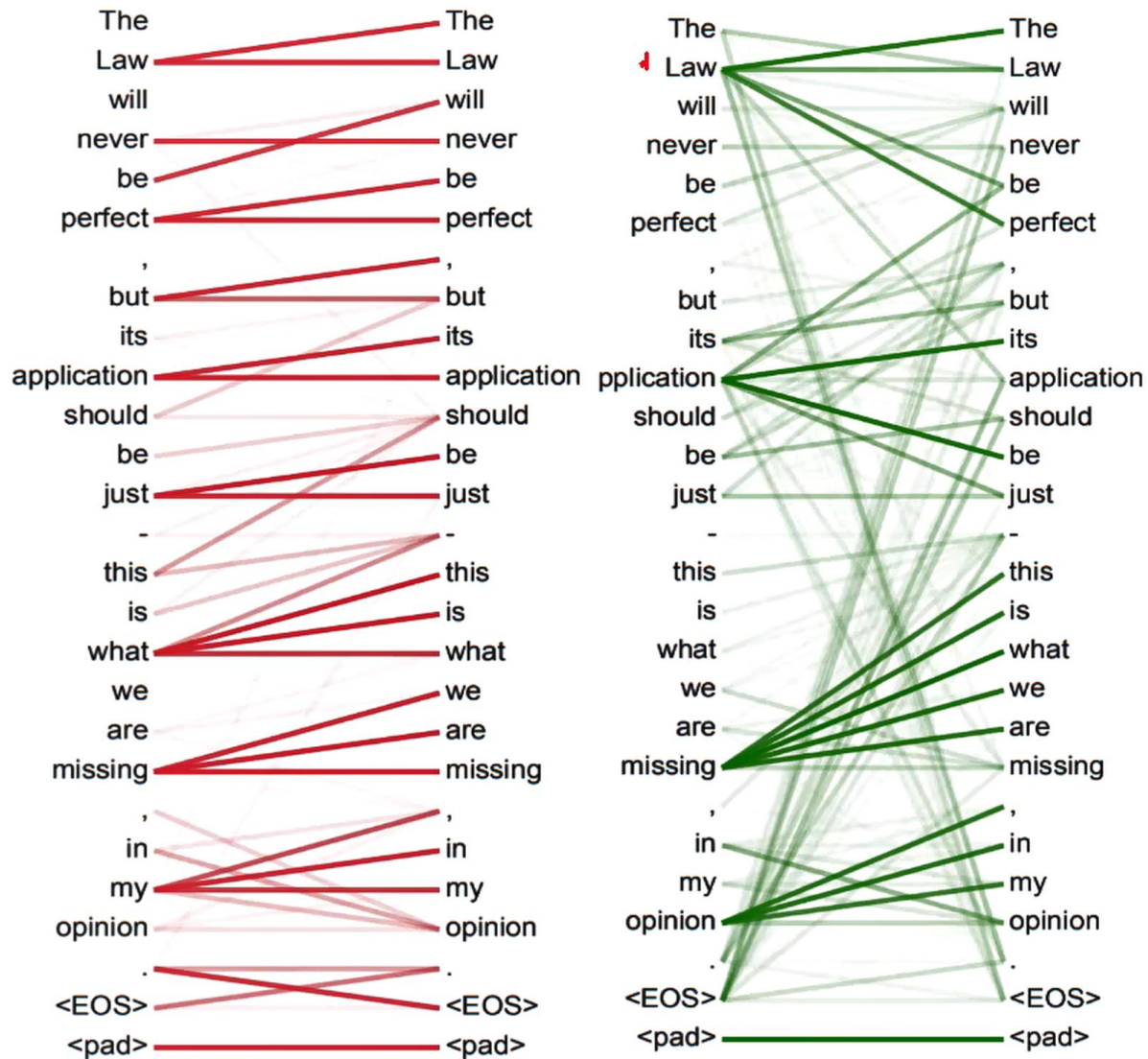




多头注意力 (Multi-Head Attention)

multi-headed结果

- 不同的注意力结果
- 得到的特征向量表达也不相同





多头注意力 (Multi-Head Attention)

```
class MultiHeadAttention(nn.Module):
    def __init__(self, heads, d_model, dropout = 0.1):
        super().__init__()

        self.d_model = d_model
        self.d_k = d_model // heads
        self.h = heads

        self.q_linear = nn.Linear(d_model, d_model)
        self.v_linear = nn.Linear(d_model, d_model)
        self.k_linear = nn.Linear(d_model, d_model)
        self.dropout = nn.Dropout(dropout)
        self.out = nn.Linear(d_model, d_model)

    def attention(q, k, v, d_k, mask=None, dropout=None):
        scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(d_k)

        # 掩盖那些为了填补长度而增加的单元, 使其通过Softmax计算后为0
        if mask is not None:
            mask = mask.unsqueeze(1)
            scores = scores.masked_fill(mask == 0, -1e9)

        scores = F.softmax(scores, dim=-1)

        if dropout is not None:
            scores = dropout(scores)

        output = torch.matmul(scores, v)
        return output

    def forward(self, q, k, v, mask=None):

        bs = q.size(0)

        # 利用线性计算划分成h个头
```

```
k = self.k_linear(k).view(bs, -1, self.h, self.d_k)
q = self.q_linear(q).view(bs, -1, self.h, self.d_k)
v = self.v_linear(v).view(bs, -1, self.h, self.d_k)

# 矩阵转置
k = k.transpose(1,2)
q = q.transpose(1,2)
v = v.transpose(1,2)

# 计算attention
scores = attention(q, k, v, self.d_k, mask, self.dropout)

# 连接多个头并输入最后的线性层
concat = scores.transpose(1,2).contiguous().view(bs, -1, self.d_model)

output = self.out(concat)

return output
```


前馈层

前馈层 (Feed-Forward Network, FFN) 接收自注意力子层的输出作为输入，并通过一个带有 ReLU 激活函数的两层全连接网络对输入进行更复杂的非线性变换。实验证明，这一非线性变换会对模型最终的性能产生重要的影响。

```
class FeedForward(nn.Module):

    def __init__(self, d_model, d_ff=2048, dropout = 0.1):
        super().__init__()

        # d_ff默认设置为2048
        self.linear_1 = nn.Linear(d_model, d_ff)
        self.dropout = nn.Dropout(dropout)
        self.linear_2 = nn.Linear(d_ff, d_model)

    def forward(self, x):
        x = self.dropout(F.relu(self.linear_1(x)))
        x = self.linear_2(x)
        return x
```

▶ 残差连接与层归一化

由Transformer 结构组成的网络结构通常都非常庞大。编码器和解码器均由很多层基本的Transformer 块组成，每一层中都包含复杂的非线性映射，这就导致模型的训练比较困难。

因此，研究人员在Transformer 块中进一步引入了**残差连接与层归一化技术**，以进一步提升训练的稳定性。具体来说，残差连接主要是指使用一条直连通道直接将对应子层的输入连接到输出，避免在优化过程中因网络过深而产生潜在的梯度消失问题：

$$\mathbf{x}^{l+1} = f(\mathbf{x}^l) + \mathbf{x}^l$$

$$\frac{\partial L}{\partial \mathbf{x}_l} = \frac{\partial L}{\partial \mathbf{x}_{l+1}} \frac{\partial \mathbf{x}_{l+1}}{\partial \mathbf{x}_l} = \frac{\partial L}{\partial \mathbf{x}_{l+1}} \left(1 + \frac{\partial f(\mathbf{x}_l)}{\partial \mathbf{x}_l} \right)$$

其中 $\mathbf{x}^l$ 表示第 l 层的输入， $f(\cdot)$ 表示一个映射函数。此外，为了使每一层的输入/输出稳定在一个合理的范围内，层归一化技术被进一步引入每个Transformer 块中：

$$LN(\mathbf{x}) = \alpha \cdot \frac{\mathbf{x} - \mu}{\sigma} + b$$



残差连接与层归一化

```
class Norm(nn.Module):  
  
    def __init__(self, d_model, eps = 1e-6):  
        super().__init__()  
  
        self.size = d_model  
  
        # 层归一化包含两个可以学习的参数  
        self.alpha = nn.Parameter(torch.ones(self.size))  
        self.bias = nn.Parameter(torch.zeros(self.size))  
  
        self.eps = eps  
  
    def forward(self, x):  
        norm = self.alpha * (x - x.mean(dim=-1, keepdim=True)) \  
        / (x.std(dim=-1, keepdim=True) + self.eps) + self.bias  
        return norm
```

$$LN(\mathbf{x}) = \alpha \cdot \frac{\mathbf{x} - \mu}{\sigma} + b$$



位置编码(Positional Encoding)

- 对于输入文本序列，先通过输入嵌入层 (Input Embedding) 将每个单词转换为其相对应的向量表示。通常，直接对每个单词创建一个向量表示。
- 由于Transformer结构不再使用基于循环的方式建模文本输入，序列中不再有任何信息能够提示模型单词之间的相对位置关系。在送入编码器端建模其上下文语义之前，一个非常重要的操作是在词嵌入中加入位置编码 (Positional Encoding)
- 序列中每一个单词所在的位置都对应一个向量。这一向量会与单词表示对应相加并送入后续模块中做进一步处理。在训练过程中，模型会自动地学习到如何利用这部分位置信息。
- 为了得到不同位置所对应的编码，Transformer 结构使用不同频率的正余弦函数，如下所示：

$$\begin{aligned} \text{PE}(\text{pos}, 2i) &= \sin\left(\frac{\text{pos}}{10000^{2i/d}}\right) \\ \text{PE}(\text{pos}, 2i + 1) &= \cos\left(\frac{\text{pos}}{10000^{2i/d}}\right) \end{aligned}$$

POS表示单词所在的位置， $2i$ 和 $2i+1$ 表示位置编码向量中的对应维度， d 则对应位置编码的总维度



位置编码(Positional Encoding)

```
class PositionalEncoder(nn.Module):
    def __init__(self, d_model, max_seq_len = 80):
        super().__init__()
        self.d_model = d_model

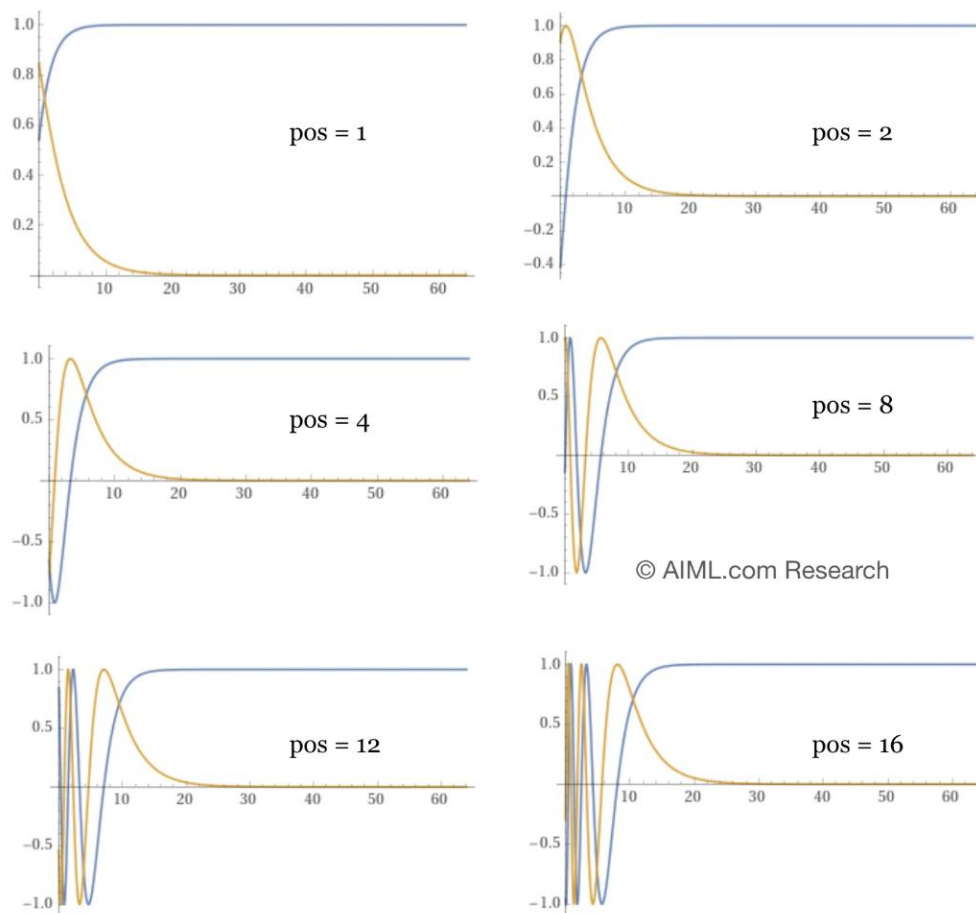
        # 根据 pos 和 i 创建一个常量 PE 矩阵
        pe = torch.zeros(max_seq_len, d_model)
        for pos in range(max_seq_len):
            for i in range(0, d_model, 2):
                pe[pos, i] = math.sin(pos / (10000 ** (i/d_model)))
                pe[pos, i + 1] = math.cos(pos / (10000 ** (i/d_model)))

        pe = pe.unsqueeze(0)
        self.register_buffer('pe', pe)

    def forward(self, x):
        # 使得单词嵌入表示相对大一些
        x = x * math.sqrt(self.d_model)
        # 增加位置常量到单词嵌入表示中
        seq_len = x.size(1)
        x = x + Variable(self.pe[:, :seq_len], requires_grad=False)
```



位置编码(Positional Encoding)



Assuming:

- Number of input tokens: 16
- Embedding Dimension: 64

64 dimensions (i) are plotted across x axis for different values of (pos), according to formula below:

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

Formula for Positional Encoding

— $\cos(pos/10000^{2i/d_{model}})$
— $\sin(pos/10000^{2i/d_{model}})$

图中展示了前 16 个输入标记的位置编码值，并使用 64 维嵌入。

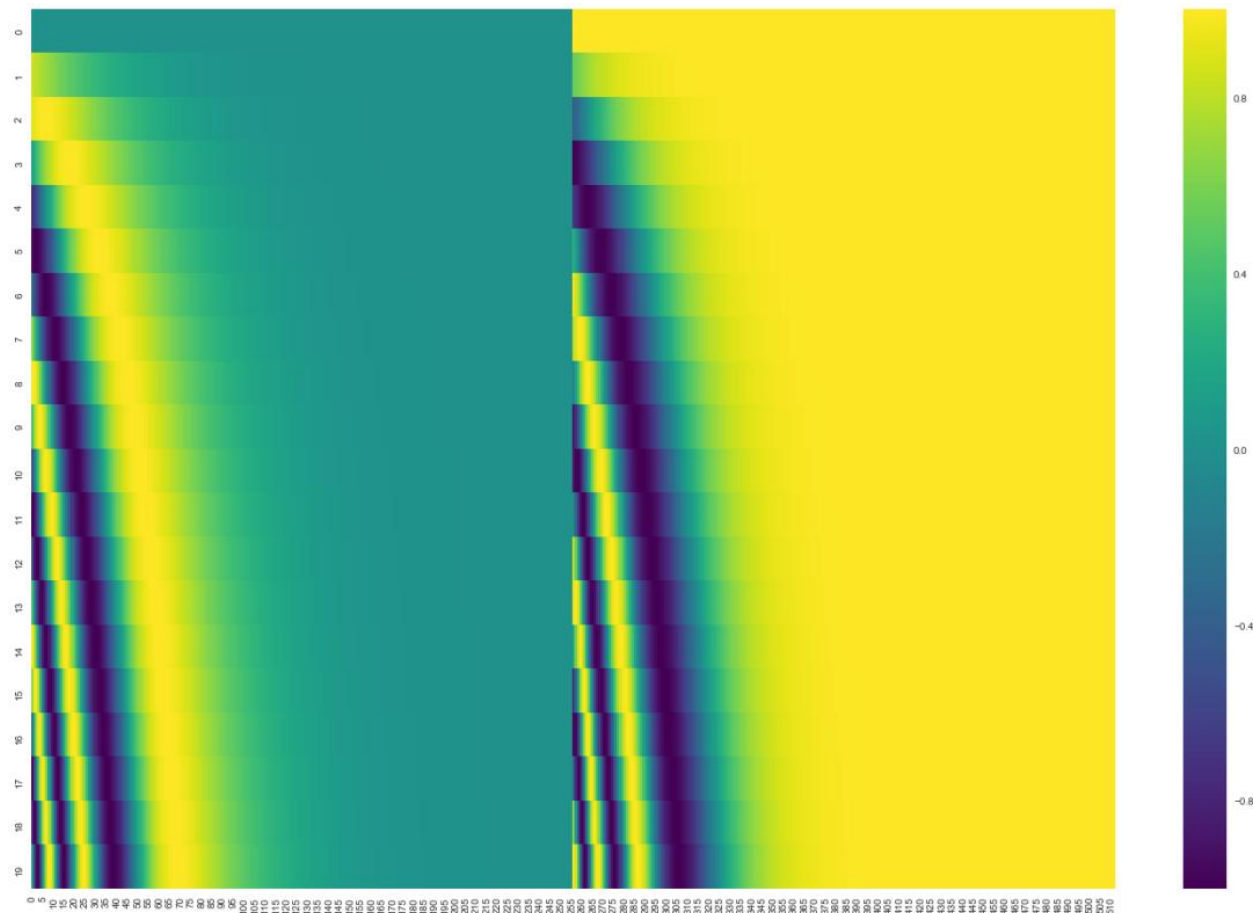
随着输入标记位置的增加，正弦和余弦周期数也随之增加，从而使模型能够理解标记的位置和顺序。



位置编码(Positional Encoding)

20字(行)的位置编码实例，词嵌入大小为512(列)。你可以看到它从中间分裂成两半。这是因为左半部分的值由一个函数(使用正弦)生成，而右半部分由另一个函数(使用余弦)生成。然后将它们拼在一起而得到每一个位置编码向量。

这个编码函数的可视化结果：



图中，每一行对应一个词向量的位置编码，所以第一行对应着输入序列的第一个词。每行包含512个值，每个值介于1和-1之间。我们已经对它们进行了颜色编码，所以图案是可见的。



位置编码(Positional Encoding)

```
class PositionalEncoder(nn.Module):
    def __init__(self, d_model, max_seq_len = 80):
        super().__init__()
        self.d_model = d_model

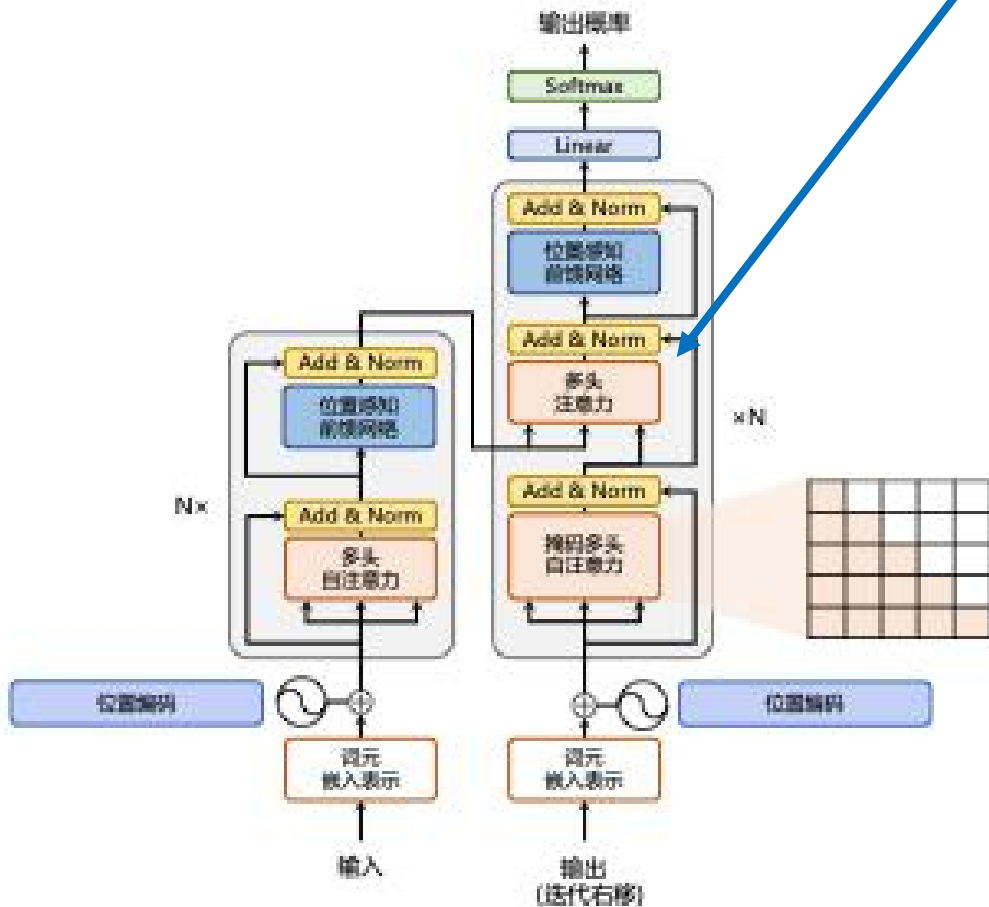
        # 根据 pos 和 i 创建一个常量 PE 矩阵
        pe = torch.zeros(max_seq_len, d_model)
        for pos in range(max_seq_len):
            for i in range(0, d_model, 2):
                pe[pos, i] = math.sin(pos / (10000 ** (i/d_model)))
                pe[pos, i + 1] = math.cos(pos / (10000 ** (i/d_model)))

        pe = pe.unsqueeze(0)
        self.register_buffer('pe', pe)

    def forward(self, x):
        # 使得单词嵌入表示相对大一些
        x = x * math.sqrt(self.d_model)
        # 增加位置常量到单词嵌入表示中
        seq_len = x.size(1)
        x = x + Variable(self.pe[:, :seq_len], requires_grad=False)
```



编码器和解码器结构



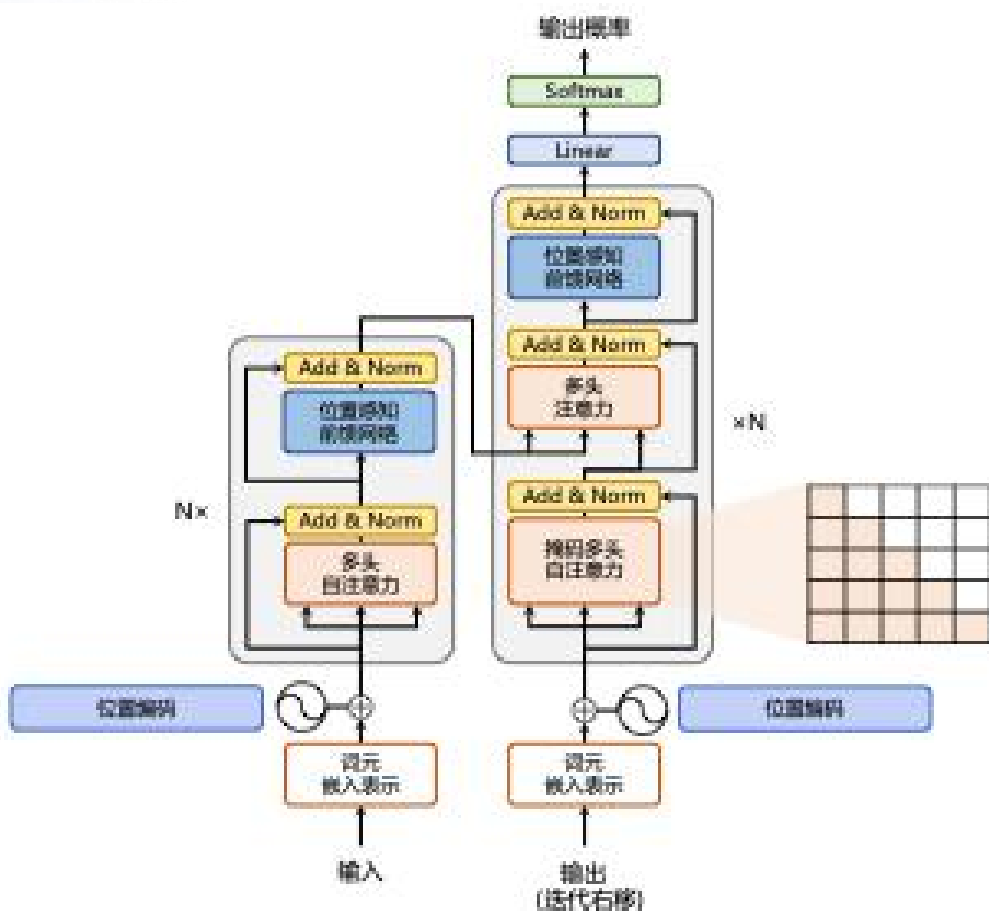
解码器端额外增加了一个**多头交叉注意力 (Multi-Head Cross-Attention)** 模块，使用交叉注意力 (Cross-Attention) 方法，查询是通过解码器前一层的输出进行投影的，而键和值是使用编码器的输出进行投影的。

其作用是在翻译的过程中，为了生成合理的目标语言序列，观测待翻译的源语言序列是什么。

基于上述编码器和解码器结构，待翻译的源语言文本，经过编码器端的每个Transformer 块对其上下文语义进行层层抽象，最终输出每一个源语言单词上下文相关的表示。

解码器端以自回归的方式生成目标语言文本，即在每个时间步 t ，根据编码器端输出的源语言文本表示，以及前 $t - 1$ 个时刻生成的目标语言文本，生成当前时刻的目标语言单词。

编码器和解码器结构



编码器端较容易实现。相比于编码器端，解码器端更复杂。具体来说，解码器的每个Transformer 块的第一个自注意力子层额外增加了注意力掩码，对应图中的掩码多头注意力（Masked Multi-Head Attention）部分

只能看到前面的词。

编码这些词

[START]

The

chef

who

[START]	The	chef	who	
		—∞	—∞	—∞
			—∞	—∞
				—∞

- 为了并行化处理，将注意力分数设为负无穷，来屏蔽模型对未来词的关注。

$$e_{ij} = \begin{cases} q_i^\top k_j, j \leq i \\ -\infty, j > i \end{cases}$$

▶ 输出

输出词汇

eos: end of sentence的缩写形式



WORD	a	am	I	thanks	student	<eos>
INDEX	0	1	2	3	4	5

我们模型的输出词表在我们训练之前的预处理流程中就被设定好。

输出

一旦我们定义了我们的输出词表，我们可以使用一个相同宽度的向量来表示我们词汇表中的每一个单词。这也被认为是一个 one-hot 编码。所以，我们可以用下面这个向量来表示单词“am”：

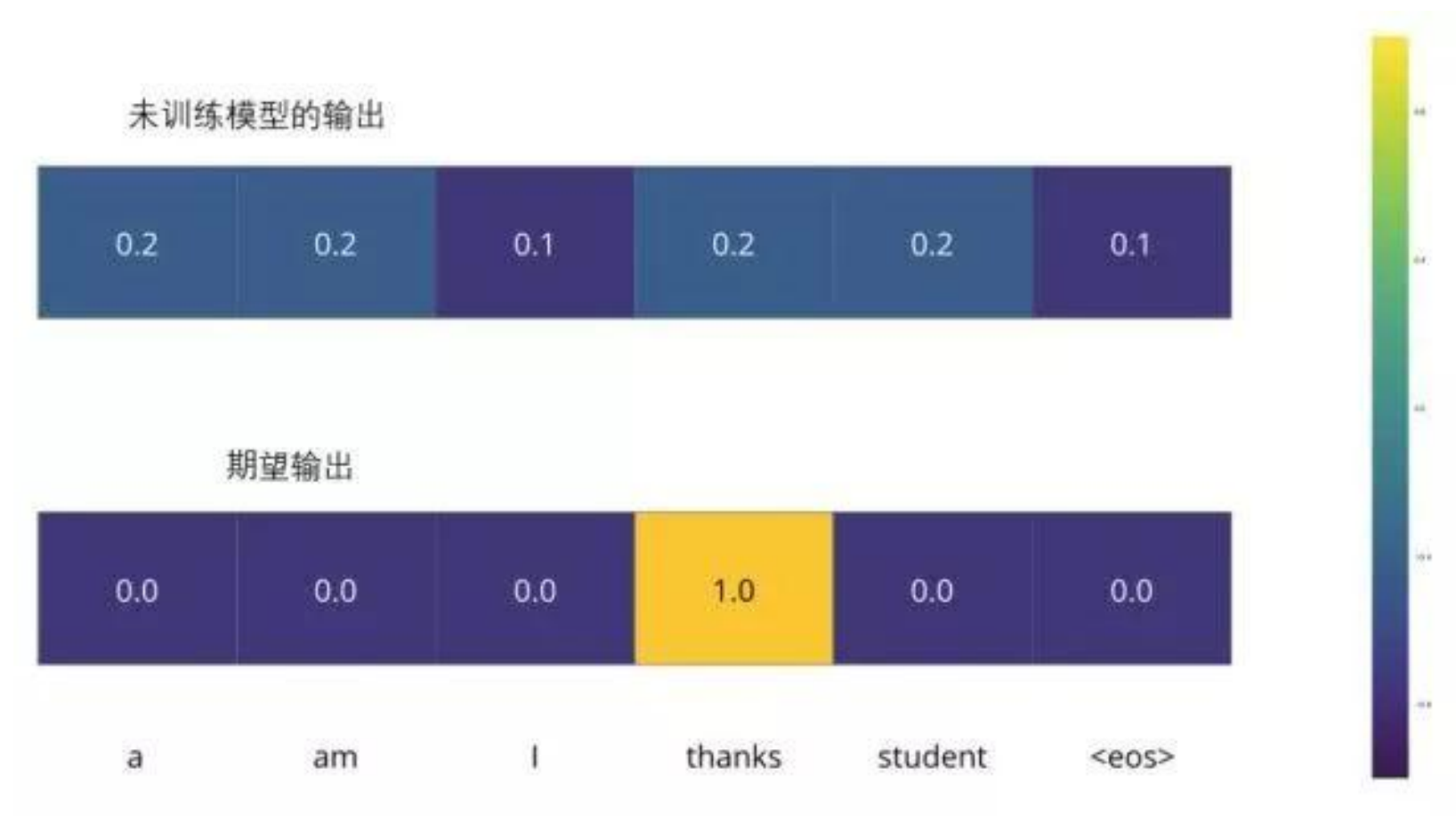
输出词表

WORD	a	am	I	thanks	student	<eos>
INDEX	0	1	2	3	4	5

单词“am”的 one-hot 编码

0.0	1.0	0.0	0.0	0.0	0.0
-----	-----	-----	-----	-----	-----

损失函数



计算输出概率分布与期望概率分布的差，一般使用cross entropy来计算。



编码器和解码器结构

编码器:

```
class EncoderLayer(nn.Module):

    def __init__(self, d_model, heads, dropout=0.1):
        super().__init__()
        self.norm_1 = Norm(d_model)
        self.norm_2 = Norm(d_model)
        self.attn = MultiHeadAttention(heads, d_model, dropout=dropout)
        self.ff = FeedForward(d_model, dropout=dropout)
        self.dropout_1 = nn.Dropout(dropout)
        self.dropout_2 = nn.Dropout(dropout)

    def forward(self, x, mask):
        attn_output = self.attn(x, x, x, mask)
        attn_output = self.dropout_1(attn_output)
        x = x + attn_output
        x = self.norm_1(x)
        ff_output = self.ff(x)
        ff_output = self.dropout_2(ff_output)
```

```
x = x + ff_output
x = self.norm_2(x)
return x
```

```
class Encoder(nn.Module):

    def __init__(self, vocab_size, d_model, N, heads, dropout):
        super().__init__()
        self.N = N
        self.embed = Embedder(vocab_size, d_model)
        self.pe = PositionalEncoder(d_model, dropout=dropout)
        self.layers = get_clones(EncoderLayer(d_model, heads, dropout), N)
        self.norm = Norm(d_model)

    def forward(self, src, mask):
        x = self.embed(src)
        x = self.pe(x)
        for i in range(self.N):
            x = self.layers[i](x, mask)
        return self.norm(x)
```



编码器和解码器结构

解码器:

```
class DecoderLayer(nn.Module):

    def __init__(self, d_model, heads, dropout=0.1):
        super().__init__()
        self.norm_1 = Norm(d_model)
        self.norm_2 = Norm(d_model)
        self.norm_3 = Norm(d_model)

        self.dropout_1 = nn.Dropout(dropout)
        self.dropout_2 = nn.Dropout(dropout)
        self.dropout_3 = nn.Dropout(dropout)

        self.attn_1 = MultiHeadAttention(heads, d_model, dropout=dropout)
        self.attn_2 = MultiHeadAttention(heads, d_model, dropout=dropout)
        self.ff = FeedForward(d_model, dropout=dropout)

    def forward(self, x, e_outputs, src_mask, trg_mask):
        attn_output_1 = self.attn_1(x, x, x, trg_mask)
        attn_output_1 = self.dropout_1(attn_output_1)
```

```
        x = x + attn_output_1
        x = self.norm_1(x)
        attn_output_2 = self.attn_2(x, e_outputs, e_outputs, src_mask)
        attn_output_2 = self.dropout_2(attn_output_2)
        x = x + attn_output_2
        x = self.norm_2(x)

        ff_output = self.ff(x)
        ff_output = self.dropout_3(ff_output)
        x = x + ff_output
        x = self.norm_3(x)

        return x
```

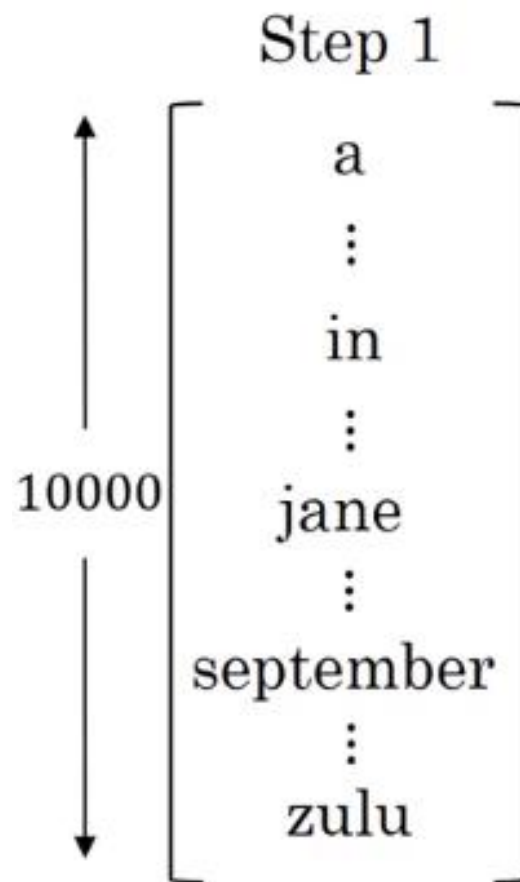
```
class Decoder(nn.Module):

    def __init__(self, vocab_size, d_model, N, heads, dropout):
        super().__init__()
        self.N = N
        self.embed = Embedder(vocab_size, d_model)
        self.pe = PositionalEncoder(d_model, dropout=dropout)
        self.layers = get_clones(DecoderLayer(d_model, heads, dropout), N)
        self.norm = Norm(d_model)

    def forward(self, trg, e_outputs, src_mask, trg_mask):
        x = self.embed(trg)
        x = self.pe(x)
        for i in range(self.N):
            x = self.layers[i](x, e_outputs, src_mask, trg_mask)
        return self.norm(x)
```

▶ 集束搜索

贪婪算法只会挑出最可能的那一个单词，然后继续。而集束搜索则会考虑多个选择，集束搜索算法会有一个参数B，叫做集束宽（beam width）。在这个例子中B=3，这样就意味着集束搜索不会只考虑一个可能结果，而是一次会考虑3个，比如对第一个单词有不同选择的可能性，最后找到in、jane、september，是英语输出的第一个单词的最可能的三个选项，然后集束搜索算法会把结果存到计算机内存里以便后面尝试用这三个词。



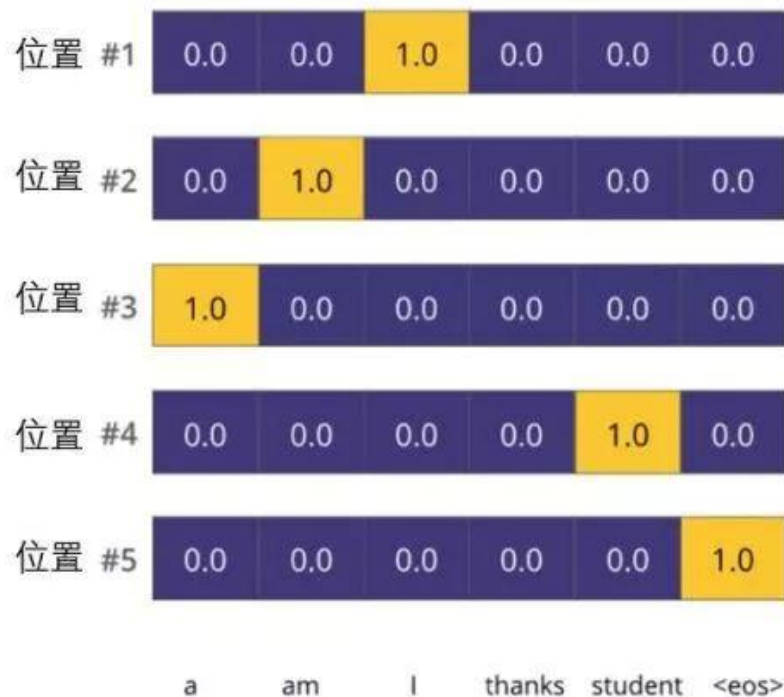
“Jane visite l'Afrique en Septembre.”（法语句子），
我们希望翻译成英语，“Jane is visiting Africa in
September”.（英语句子）



训练结果

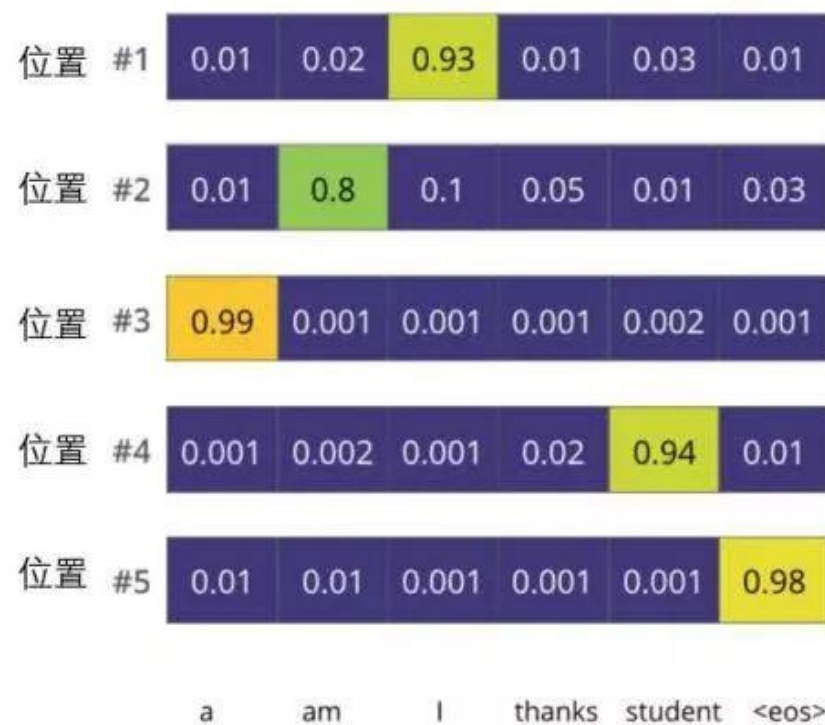
模型的目标输出

输出词表： a am I thanks student <eos>



模型训练后的输出

输出词表： a am I thanks student <eos>



Transformer

区别	RNN	Transformer
区别	RNN	Transformer
序列处理方式	循环处理、逐个处理序列中的元素	并行处理，一次性查看所有元素
捕捉长期依赖关系	随着序列长度的增加，容易出现梯度消失或梯度爆炸的问题，难以有效地捕捉长期依赖	能够直接建立序列中任意两个位置之间的连接，无论它们在序列中距离多远
计算效率	串行计算，效率较低	并行计算，效率更高
可解释性	相对较差	注意力机制提供一定的可解释性（可以可视化注意力权重）
主要应用（早期）	机器翻译、语音识别、文本生成等	自然语言处理（NLP）的各种任务，图像识别，语音识别等
主要应用（现在）	一些时间序列预测，语音识别等领域仍有应用，但逐渐被 Transformer 取代	成为现代 NLP 的主流架构，广泛应用于各种序列任务

03

Transformer改进

2025



1.稀疏注意力机制

对一些训练好的Transformer 结构中的注意力矩阵进行分析时发现，其中很多是稀疏的，因此可以通过限制Query-Key 对的数量来降低计算复杂度。这类方法称为稀疏注意力（SparseAttention）机制。

稀疏注意力机制的核心思想是在自注意力计算中引入稀疏性，即不是让序列中的每个位置都与其他所有位置进行注意力计算，而是仅选择部分位置进行计算。

根据确定稀疏连接的度量标准，将这些方法分为两类：基于位置的稀疏注意力和基于内容的稀疏注意力。



1.稀疏注意力机制

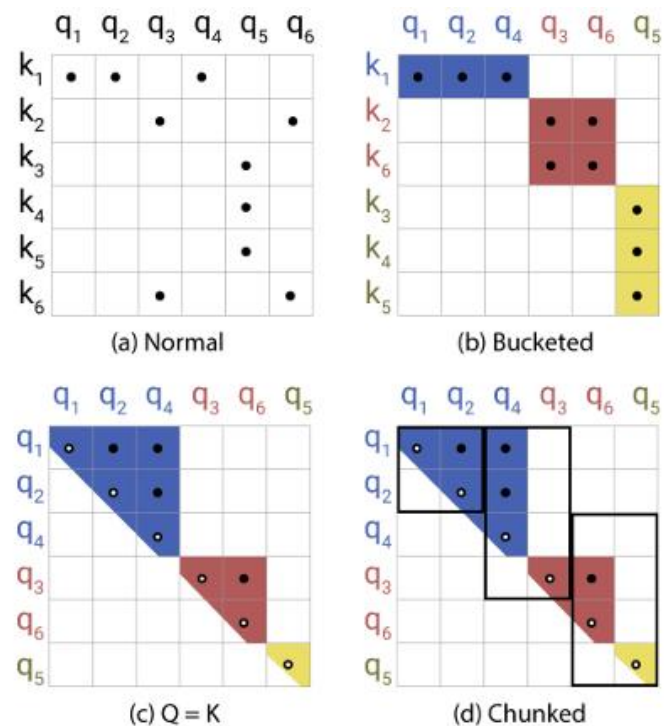
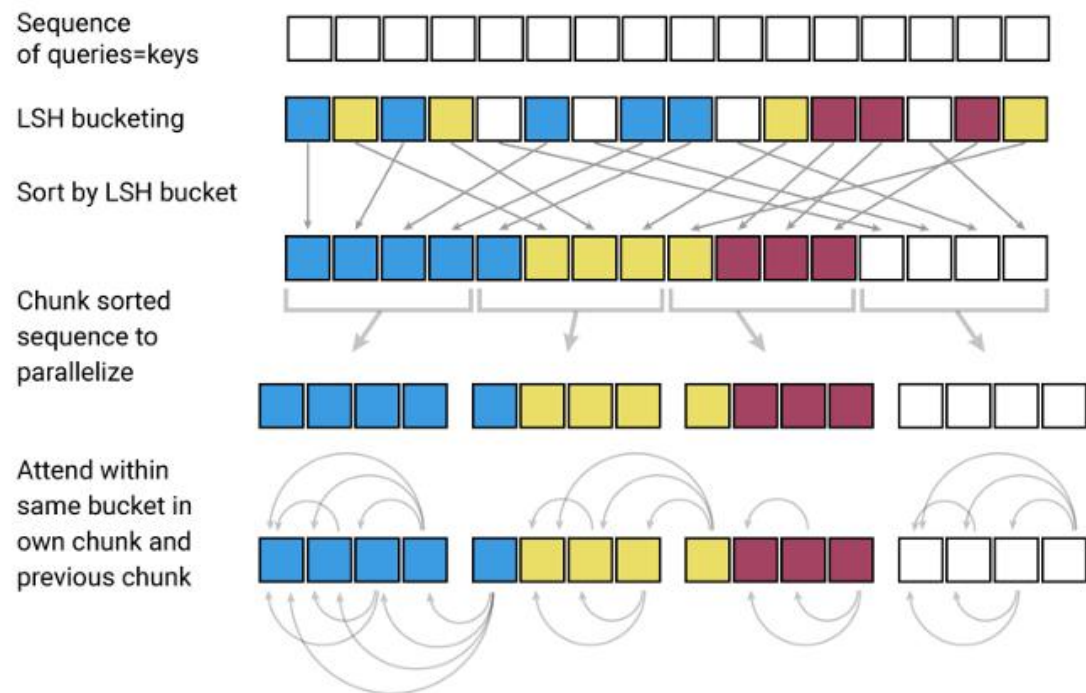
基于内容的稀疏注意力机制根据输入数据创建稀疏注意力，其中一种很简单的方法是选择和给定查询（Query）有很高相似度的键（Key）。

Routing Transformer 是一种改进的 Transformer 模型，通过引入一种称为“路由”的机制来解决这个问题，从而提高处理长序列的效率。采用K-means 聚类方法，针对Query和Key进行聚类，类中心向量集合为 $\{\mu_i\}_{i=1}^k$ ，其中k是类中心的个数。每个Query 只与其处在相同簇（Cluster）下的Key 进行交互。中心向量采用滑动平均的方法进行更新：

$$\begin{aligned}\tilde{\mu} &\leftarrow \lambda \tilde{\mu} + (1 - \lambda) \left(\sum_{i:\mu(\mathbf{q}_i)=\mu} \mathbf{q}_i + \sum_{j:\mu(\mathbf{k}_j)=\mu} \mathbf{k}_j \right) \\ c_\mu &\leftarrow \lambda c_\mu + (1 - \lambda) |\mu| \\ \mu &\leftarrow \frac{\tilde{\mu}}{c_\mu}\end{aligned}$$

1. 稀疏注意力机制

Reformer旨在解决原始Transformer模型处理长序列数据时的计算和存储效率问题。主要通过引入局部敏感哈希（LSH）技术来对输入序列进行分桶(bucketing), 使得每个元素只需要与同一桶内的元素计算注意力得分。并且通过使用可逆残差层设计允许模型在反向传播时重建前向传播的中间激活值, 而无需显式地存储它们, 从而大幅减少了内存需求。



1. 稀疏注意力机制

Global Attention机制通过添加一些全局节点来工作，这些节点可被看作信息传播枢纽。这些全局节点有能力关注序列中的每一个节点，这意味着它们可以捕获和累积整个序列的信息。同时，序列中的每个节点也都会关注这些全局节点。

这样，即使是序列中相距很远的节点，它们之间的信息也可以通过这些全局节点进行有效传播。通过这种方式，**全局节点充当了一个信息集散地**，帮助远距离的节点间接地相互“通信”。这样，即使在稀疏注意力的框架下，模型也能更好地捕捉长距离依赖性，从而提高了模型对序列数据的理解能力。

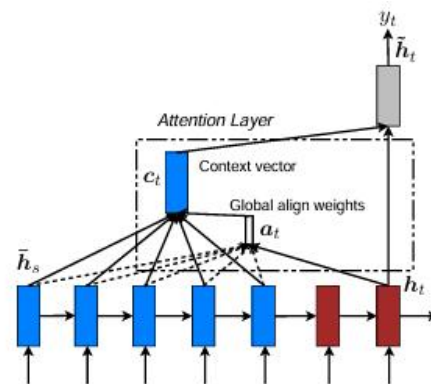
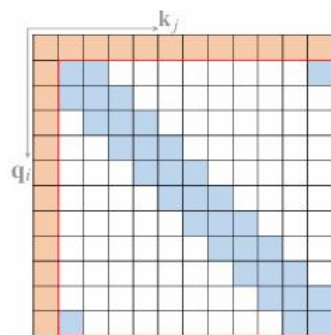


Figure 2: **Global attentional model** – at each time step t , the model infers a *variable-length* alignment weight vector a_t based on the current target state h_t and all source states $\bar{h}_s$. A global context vector c_t is then computed as the weighted average, according to a_t , over all the source states.

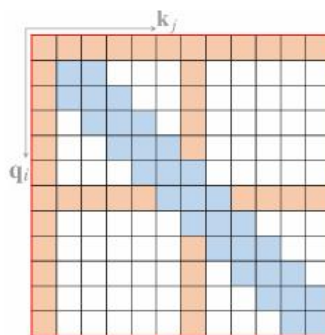
1. 稀疏注意力机制

Band Attention（也被称为滑动窗口注意力或局部注意力）是一种在处理数据时考虑数据的局部性质的注意力机制。

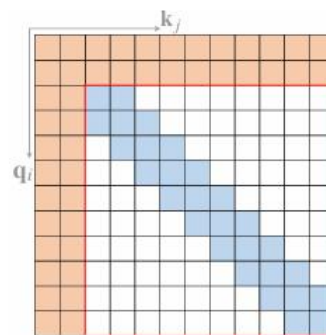
这种注意力通过将注意力限制在一个局部区域内，带状注意力机制能够显著减少计算量，因为它只计算和存储那些可能高度相关的元素对之间的关系。此外，这种方法还能帮助模型更加聚焦于局部上下文，这在很多任务中是非常有益的，比如在处理语言时捕捉短语层面的依赖关系，或在处理图像时关注局部特征。



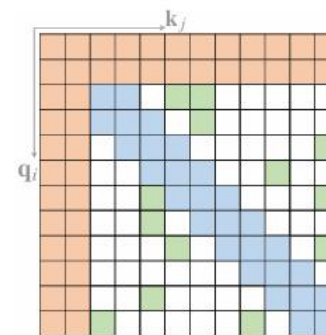
(a) Star-Transformer



(b) Longformer



(c) ETC



(d) BigBird

2. FlashAttention

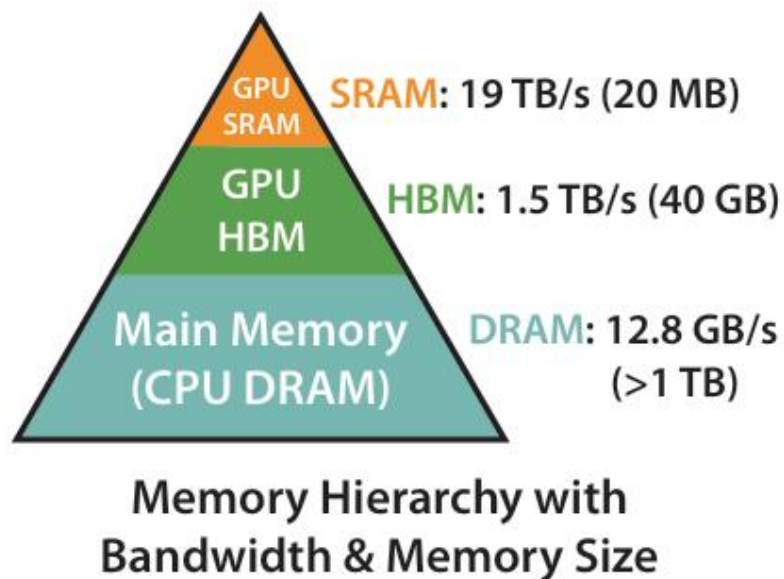
FlashAttention旨在加速注意力计算并减少内存占用。FlashAttention利用底层硬件的内存层次知识，例如GPU的内存层次结构，来提高计算速度和减少内存访问开销。FlashAttention的核心原理是通过将输入分块并在每个块上执行注意力操作，从而减少对高带宽内存（HBM）的读写操作。



2. FlashAttention

Flash Attention 的动机是尽可能避免大尺寸的注意力权重矩阵在 HBM 和 SRAM 之间的换入换出。具体方法包含两个部分：tiling 和 recomputation。

其中，HBM是一种高带宽内存接口，用于3D堆叠的SDRAM，具有较高的带宽和较低的功耗。SRAM是一种静态随机访问存储器，用于高速缓存等内部存储器，具有更快的访问速度和更低的延迟，但成本更高且占用更多芯片空间。

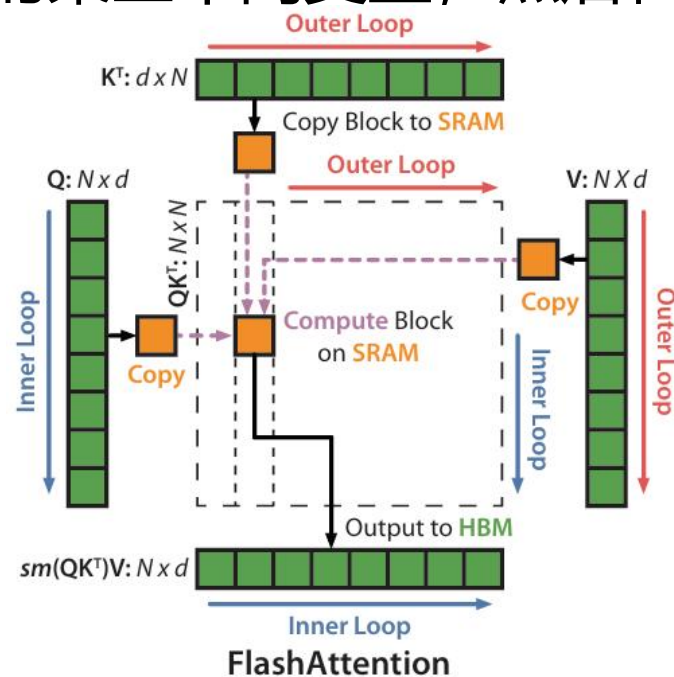


GOU A100的内存分布

2. FlashAttention

tiling 的基本思路：不直接对整个输入序列计算注意力，而是将其分为多个较小的块，逐个对这些块进行计算，增量式地进行 softmax 的规约。规约过程中只需要更新某些中间变量，不需要计算整个注意力权重矩阵。

recomputation 的基本思路：基于 tiling 技巧，在反向传播过程中不保留整个注意力权重矩阵，而是只保留前向过程中 tiling 的某些中间变量，然后在反向传播过程中重新计算注意力权重矩阵。





2.FlashAttention

代码 2.1: FlashAttention 算法

输入: $Q, K, V \in \mathbb{R}^{N \times d}$ 位于高速显存 (HBM) 中, GPU 芯片中的 SRAM 大小为 M

输出: O

$B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$ // 设置块大小 (block size)

在 HBM 中初始化 $O = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $l = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$

将矩阵 Q 切分成 $T_r = \lceil \frac{M}{B_r} \rceil$ 块 $Q_1, \dots, Q_{T_r}$, $Q_i \in \mathbb{R}^{B_r \times d}$

将矩阵 K 切分成 $T_c = \lceil \frac{M}{B_c} \rceil$ 块 $K_1, \dots, K_{T_c}$, $K_i \in \mathbb{R}^{B_c \times d}$

将矩阵 V 切分成 T_c 块 $V_1, \dots, V_{T_c}$, $V_i \in \mathbb{R}^{B_c \times d}$

将矩阵 O 切分成 T_r 块 $O_1, \dots, O_{T_r}$, $O_i \in \mathbb{R}^{B_r \times d}$

将 l 切分成 T_r 块 $l_1, \dots, l_{T_r}$, $l_i \in \mathbb{R}^{B_r}$

将 m 切分成 T_r 块 $m_1, \dots, m_{T_r}$, $m_i \in \mathbb{R}^{B_r}$

for $j = 1$ to T_c do

 将 K_j 和 V_j 从 HBM 中读入芯片存储 SRAM

 for $i = 1$ to T_r do

 计算 $S_{ij} = Q_i K_j^T \in \mathbb{R}^{B_r \times B_c}$

 计算 $\tilde{m}_{ij} = \text{rowmax}(S_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{P}_{ij} = \exp(S_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$

 计算 $\tilde{l}_{ij} = \text{rowsum}(\tilde{P}_{ij}) \in \mathbb{R}^{B_r}$

 计算 $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $l_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} l_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{l}_{ij} \in \mathbb{R}^{B_r}$

 将 $O \leftarrow \text{diag}(l_i^{\text{new}})^{-1} (\text{diag}(l_i) e^{m_i - m_i^{\text{new}}} O_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{P}_{ij} V_j)$ 写回 HBM 中

 将 $l_i \leftarrow l_i^{\text{new}}$ 和 $m_i \leftarrow m_i^{\text{new}}$ 写回 HBM 中

 end

end

return O

04

总结与讨论

2025

2025

谢谢大家

时间：202X.X